CO502 – Advanced Computer Architecture

Lab Report

Lab Parts 1, 2, 3, 4

Group 04
E/17/083, E/19/455

Contents

I Lab Part 1 – Planning **6**

1 Activity 01: RV32I Base Instruction Set **6**
1.1 U-Type Instructions (LUI, AUIPC) . . . 6
1.1.1 LUI (Load Upper Immediate) . . . 6
1.1.2 AUIPC (Add Upper Immediate to PC) . . . 6
1.2 J-Type Instructions . . . 6
1.2.1 JAL (Jump and Link) . . . 6
1.3 I-Type Instructions . . . 7
1.3.1 JALR (Jump and Link Register) . . . 7
1.3.2 Load Instructions . . . 7
1.3.3 Immediate Instructions . . . 7
1.4 B-type (Branch Instructions) . . . 8
1.5 S-type (Store Instructions) . . . 9
1.6 R-type (Register-Register ALU) . . . 9
1.7 FENCE, ECALL, EBREAK . . . 10
1.7.1 FENCE . . . 10
1.7.2 ECALL / EBREAK . . . 10

2 RV32M Multiply and Divide Instructions **10**
2.1 MUL – Multiply (Low 32 bits, signed) . . . 11
2.2 MULH – Multiply High (signed × signed) . . . 11
2.3 MULHSU – Multiply High (signed × unsigned) . . . 11
2.4 MULHU – Multiply High (unsigned × unsigned) . . . 11
2.5 DIV – Divide (signed) . . . 11
2.6 DIVU – Divide (Unsigned) . . . 11
2.7 REM – Remainder (Signed) . . . 11
2.8 REMU – Remainder (Unsigned) . . . 12

3 Activity 2 **12**
3.1 Hardware Units . . . 12
3.2 Control Signals . . . 12

4 Activity 3 **13**

II Lab Part 2 – Hardware Units **13**

5 Fetch Cycle **14**
5.1 Program Counter . . . 15
5.1.1 Design Choices . . . 15
5.1.2 Limitations . . . 15
5.1.3 Sample Waveform . . . 15
5.2 Instruction Memory . . . 15
5.2.1 Design Choices . . . 16
5.2.2 Limitations . . . 16
5.2.3 Sample Waveform . . . 16
5.3 Use of Two Multiplexers in Fetch Stage . . . 16
5.3.1 Design Choices . . . 17
5.3.2 Limitations . . . 17
5.4 32-bit Adder Module . . . 17
5.4.1 Design Choices . . . 17
5.4.2 Limitations . . . 17
5.4.3 Sample Waveform . . . 17

6 Decode Stage **18**
6.1 Control Unit . . . 19
6.1.1 Design Choices . . . 20
6.1.2 Limitations . . . 20
6.1.3 Sample Waveform . . . 20
6.2 Register Files . . . 21
6.2.1 Design Choices . . . 21
6.2.2 Limitations . . . 21
6.2.3 Sample Waveform . . . 22
6.3 Immediate Generator . . . 22
6.3.1 I-type Immediate . . . 22
6.3.2 S-type Immediate . . . 22
6.3.3 B-type Immediate . . . 22
6.3.4 U-type Immediate . . . 22
6.3.5 J-type Immediate . . . 23
6.3.6 Default Case . . . 23
6.3.7 Design Choices . . . 23
6.3.8 Limitations . . . 23
6.3.9 Sample Waveform . . . 23

7 Execution Stage **25**
7.1 Design Overview . . . 26
7.2 ALU (Arithmetic & Logical Unit) . . . 26
7.2.1 ALU Design Summary . . . 26
7.3 Btype – Design Summary . . . 26
7.4 Jtype – Design Summary . . . 27
7.5 LoadStoreUnit – Design Summary . . . 27
7.6 muldiv – Design Summary . . . 28
7.7 RItype: ALU Logic for R-type and I-type Instructions . . . 29
7.8 Utype: ALU Logic for U-type Instructions . . . 29
7.9 StoreConverter Module . . . 29
7.9.1 Sample Waveform . . . 30

8 Memory Stage **30**
8.1 Data Memory . . . 31
8.1.1 Key Features . . . 31
8.1.2 Limitations . . . 31
8.1.3 Sample Waveform . . . 31

9 WriteBack Stage **33**
9.1 Design Overview . . . 34
9.1.1 Key Modules . . . 34
9.2 LoadConverter Module . . . 34
9.2.1 Data Conversion . . . 34
9.2.2 Design Choices . . . 34
9.2.3 Limitations . . . 34
9.2.4 Sample Waveform . . . 35

III Lab Part 3 – Integration **35**

10 Integration of Each Stage **35**
10.1 Instruction Fetch Stage Integration . . . 35
10.1.1 Functional Overview . . . 35
10.1.2 Design Choices . . . 36
10.1.3 Limitations . . . 36
10.2 Instruction Decode (ID) Stage – Integration . . . 37
10.2.1 Submodule Integration . . . 37
10.2.2 Limitations . . . 38
10.3 Execution Stage Integration . . . 39
10.3.1 Design Objectives . . . 40
10.3.2 Integrated Modules . . . 40
10.3.3 Design Choices . . . 40
10.3.4 Limitations . . . 40
10.4 Integration of Memory Stage . . . 40
10.4.1 Key Signals Passed . . . 40
10.4.2 Limitations . . . 40
10.5 Integration Summary of the WriteBack Stage . . . 41
10.5.1 Key Inputs (from MEM/WB register) . . . 42

10.5.2 Internal Units 42
10.5.3 Key Outputs 42

11 Design Overview of Inter-Stage Pipeline Registers **42**
11.0.1 Design Choices 42
11.0.2 Limitations 42

12 Testing for Instructions after Overall Integration **43**
12.1 I-type Instructions 43
12.2 R-type Instructions 44
12.3 Multiplication and Division Instructions 45
12.4 Branch Instructions 47
12.5 Store Instructions 50
12.6 Load Instructions 50
12.7 U-type Instructions 50
12.8 Jump Instructions 51

IV Lab Part 4 – Hazard Handling **51**

13 Introduction **52**
13.1 Data Hazards 52
13.2 Control Hazards 53
13.3 Structural Hazards 54
13.3.1 Causes 54
13.3.2 Remedies and Solutions 54

14 Forwarding Unit **55**
14.1 How It Works 55
14.2 Result After Forwarding (Example 1) 55

15 Hazard Detection Unit **56**
15.1 How It Works 56
15.2 Result 56
15.3 Testing with Load-Use Hazard Example 56

16 Control Hazard Unit **57**
16.1 Purpose 57
16.2 How It Works 58
16.3 Outputs 58
16.4 Testing with Control Hazard Example 58

Part I
Lab Part 1 – Planning

1 Activity 01: RV32I Base Instruction Set

1.1 U-Type Instructions (LUI, AUIPC)

1.1.1 LUI (Load Upper Immediate)

- **Opcode:** `0110111`
- **Format:** `imm[31:12] | rd | opcode`
- **Purpose:** Load a 20-bit constant into the upper 20 bits of a register.
- **Operation:** `rd = imm << 12`
- **Use Case:** Efficiently loads large constants into a register.
- **Example:** `lui x5, 0x12345` loads `0x12345000` into `x5`.

1.1.2 AUIPC (Add Upper Immediate to PC)

- **Opcode:** `0010111`
- **Format:** `imm[31:12] | rd | opcode`
- **Purpose:** Generate PC-relative addresses.
- **Operation:** `rd = PC + (imm << 12)`
- **Use Case:** Useful for position-independent code, loading addresses.
- **Example:** `auipc x1, 0x2` at PC `0x1000` → `x1 = 0x1000 + 0x2000 = 0x3000`

1.2 J-Type Instructions

1.2.1 JAL (Jump and Link)

- **Opcode:** `1101111`
- **Format:** `imm[20|10:1|11|19:12] | rd | opcode`
- **Purpose:** Unconditional jump with return address.
- **Operation:** `rd = PC + 4; PC += imm`
- **Use Case:** Function calls, long-distance jumps.
- **Note:** Immediate is signed and aligned (LSB is 0).
- **Example:** `jal x1, 256` saves return address and jumps ahead by 256 bytes.

1.3 I-Type Instructions

1.3.1 JALR (Jump and Link Register)

- **Opcode:** `1100111`
- **Format:** `imm[11:0] | rs1 | 000 | rd | opcode`
- **Purpose:** Jump to address in register + offset.
- **Operation:** `rd = PC + 4; PC = (rs1 + imm) & ~1`
- **Use Case:** Function returns, indirect jumps.
- **Example:** `jalr x0, 0(x1)` → `PC = x1` (returns to address in `x1`)

1.3.2 Load Instructions

- **Opcode:** `0000011`

Instruction	funct3	Description
LB	000	Load signed byte
LH	001	Load signed half-word
LW	010	Load word
LBU	100	Load unsigned byte
LHU	101	Load unsigned half word

Table 1: Load Instructions

- **Format:** `imm[11:0] | rs1 | funct3 | rd | opcode`
- **Purpose:** Read from memory into a register.
- **Operation:** `rd = Mem[rs1 + imm]` (with sign or zero extension depending on instruction)

1.3.3 Immediate Instructions

- **Opcode:** `0010011`

Instruction	funct3	Operation	Description
ADDI	000	rd = rs1 + imm	Adds a signed 12-bit immediate to the value in register rs1 and stores the result in rd
SLTI	010	rd = (rs1 ¡ imm) ? 1 : 0	Compares rs1 and the immediate value as signed integers
SLTIU	011	rd = (unsigned(rs1) ¡ unsigned(imm)) ? 1 : 0	Like SLTI, but treats both operands as unsigned integers
XORI	100	rd = rs1 ˆ imm	Performs a bitwise XOR between rs1 and the immediate
ORI	110	rd = rs1 — imm	Performs a bitwise OR between rs1 and the immediate
ANDI	111	rd = rs1 & imm	Performs a bitwise AND between rs1 and the immediate

Table 2: Immediate Instructions

- **Format:** `imm[11:0] | rs1 | funct3 | rd | opcode`

Instruction	Funct3	Funct7	Operation	Description
SLLI	001	0000000	rd = rs1 ¡¡ shamt	Shifts rs1 left by shamt bits, filling with zeros
SRLI	101	0000000	rd = rs1 ¿¿ shamt (logical, zero-fill)	Shifts rs1 right by shamt bits, filling the left with zeros (logical shift)
SRAI	101	0100000	rd = rs1 ¿¿ shamt (arithmetic, sign-extend)	Shifts rs1 right by shamt bits, preserving the sign (copying the sign bit)

Table 3: Shift Immediate Instructions

Shift Immediate (opcode 0010011, funct7 and funct3 used)

- **Format:** `funct7 | shamt | rs1 | funct3 | rd | opcode`

1.4 B-type (Branch Instructions)

- **Opcode:** `1100011`

Instruction	funct3	Description
BEQ	000	Branch if equal
BNE	001	Branch if not equal
BLT	100	Branch if less than (signed)
BGE	101	Branch if greater or equal
BLTU	110	Branch if less (unsigned)
BGEU	111	Branch if greater (unsigned)

Table 4: Branch Instructions

- **Format:** `imm[12|10:5] | rs2 | rs1 | funct3 | imm[4:1|11] | opcode`
- **Operation:** If condition is true, `PC = PC + imm`

1.5 S-type (Store Instructions)

- **Opcode:** `0100011`

Instruction	funct3	Description
SB	000	Store byte
SH	001	Store half-word
SW	010	Store word

Table 5: Store Instructions

- **Format:** `imm[11:5] | rs2 | rs1 | funct3 | imm[4:0] | opcode`
- **Operation:** `Mem[rs1 + imm] = rs2`

1.6 R-type (Register-Register ALU)

- **Opcode:** `0110011`

Instruction	funct3	funct7	Operation
ADD	000	0000000	rd = rs1 + rs2
SUB	000	0100000	rd = rs1 - rs2
SLL	001	0000000	rd = rs1 ¡¡ rs2[4:0]
SLT	010	0000000	rd = (rs1 ¡ rs2) (signed)
SLTU	011	0000000	rd = (rs1 ¡ rs2) (unsigned)
XOR	100	0000000	rd = rs1 ˆ rs2
SRL	101	0000000	Logical right shift
SRA	101	0100000	Arithmetic right shift
OR	110	0000000	rd = rs1 — rs2
AND	111	0000000	rd = rs1 & rs2

Table 6: R-type Instructions

- **Format:** `funct7 | rs2 | rs1 | funct3 | rd | opcode`

1.7 FENCE, ECALL, EBREAK

1.7.1 FENCE

- **Opcode:** `0001111`
- **Format:** `fm | pred | succ | rs1=0 | funct3=000 | rd=0 | opcode`
- **Purpose:** Memory and I/O ordering.
- **Operation:** `rd = imm << 12`
- Used in multi-core or I/O situations to enforce memory operation order.

1.7.2 ECALL / EBREAK

- **Opcode:** `1110011`
- **Format:** `000000000000 | 00000 | 000 | 00000 | opcode`
- **ECALL:** Environment call. Used for syscall interface.
- **EBREAK:** Causes a breakpoint trap.

2 RV32M Multiply and Divide Instructions

All M instructions have R-type instruction format: `funct7 | rs2 | rs1 | funct3 | rd | opcode`

2.1 MUL – Multiply (Low 32 bits, signed)

- **Opcode:** `0110011`, **funct7:** `0000001`, **funct3:** `000`
- **Purpose:** Signed × Signed, return lower 32 bits
- **Operation:** `rd = (int32_t)rs1 * (int32_t)rs2`
- **Use Case:** General fast signed multiplication
- **Example:** `MUL x5, x2, x3` → `x5 = x2 * x3` (low 32 bits only)

2.2 MULH – Multiply High (signed × signed)

- **Opcode:** `0110011`, **funct7:** `0000001`, **funct3:** `001`
- **Purpose:** Signed × Signed, return upper 32 bits
- **Operation:** `rd = ((int64_t)rs1 * (int64_t)rs2) >> 32`
- **Use Case:** When full 64-bit result is needed.

2.3 MULHSU – Multiply High (signed × unsigned)

- **Opcode:** `0110011`, **funct7:** `0000001`, **funct3:** `010`
- **Operation:** `rd = ((int64_t)rs1 * (int64_t)rs2) >> 32`

2.4 MULHU – Multiply High (unsigned × unsigned)

- **Opcode:** `0110011`, **funct7:** `0000001`, **funct3:** `011`
- **Purpose:** Unsigned × Unsigned, return upper 32 bits
- **Operation:** `rd = ((uint64_t)rs1 * (uint64_t)rs2) >> 32`

2.5 DIV – Divide (signed)

- **Opcode:** `0110011`, **funct7:** `0000001`, **funct3:** `100`
- **Operation:** `rd = (int32_t)rs1 / (int32_t)rs2`

2.6 DIVU – Divide (Unsigned)

- **Opcode:** `0110011`, **funct7:** `0000001`, **funct3:** `101`
- **Operation:** `rd = (uint32_t)rs1 / (uint32_t)rs2`

2.7 REM – Remainder (Signed)

- **Opcode:** `0110011`, **funct7:** `0000001`, **funct3:** `110`
- **Operation:** `rd = (int32_t)rs1 % (int32_t)rs2`

2.8 REMU – Remainder (Unsigned)

- **Opcode:** `0110011`, **funct7:** `0000001`, **funct3:** `111`
- **Operation:** `rd = (uint32_t)rs1 % (uint32_t)rs2`

3 Activity 2

3.1 Hardware Units

- Arithmetic and Logical Unit
- Control Unit
- Register File
- Instruction Memory
- Data Memory
- Immediate Generator
- Program Counter
- Multiplexers
- Adders
- Shifters
- StoreConverter
- LoadConverter

3.2 Control Signals

- **ALUSelect** – Signal to do selections inside ALU when operating
- **ImmSelect** – Select whether Immediate value or Register output
- **PCSelect** – Select whether Program Counter Value or Register Output
- **Jtype** – 1 if a J-type instruction is operated
- **MemRead** – Whether data is read from data memory
- **MemWrite** – Whether data is written to Data Memory
- **RegWrite** – Whether Data is written to register file

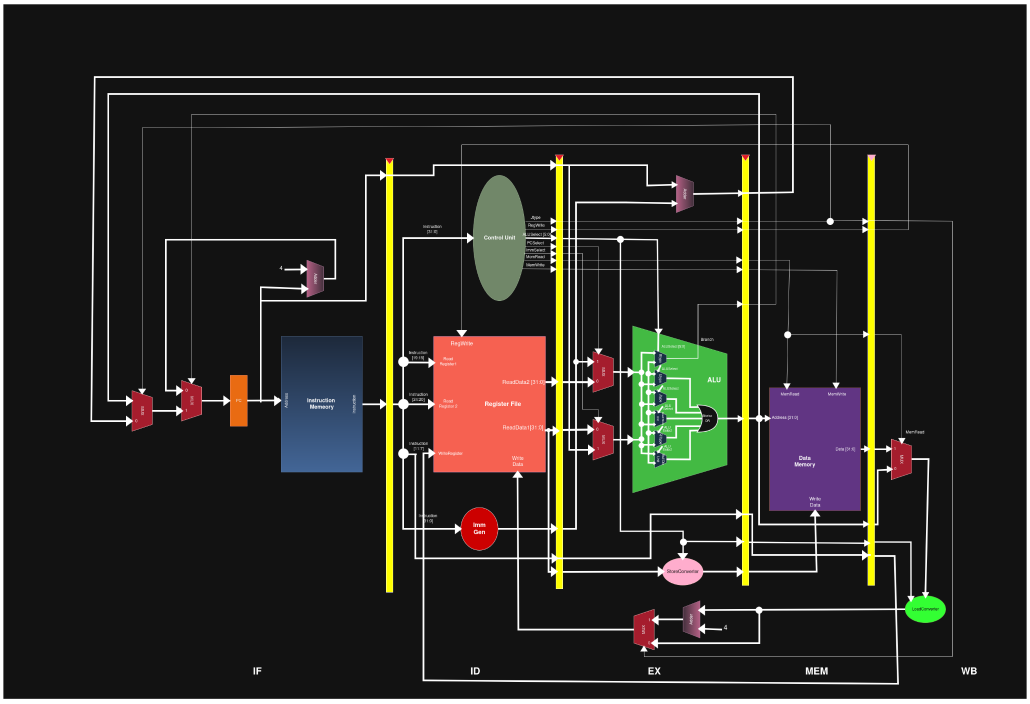


Figure 1: Full RISC-V Pipelined Processor Architecture (IF, ID, EX, MEM, WB)

4 Activity 3

- Tests to verify functionality of components
- Test fetching instructions from memory based on the program counter
- Test updating the program counter according to branch and jump instructions
- Test proper decoding of different instruction types (R-type, I-type, S-type, B-type, U-type, J-type)
- Test correct reading of operands from the register file
- Test correct results of arithmetic and logical operations
- Test proper loading and storing of data from/to memory
- Tests writing results to the register file

Part II
Lab Part 2 – Hardware Units

The RISC-V pipelined processor is organized into five stages: Fetch, Decode, Execute, Memory, and Writeback, each handling a specific part of instruction processing. In the Fetch stage, the instruction is retrieved from memory using the program counter. The Decode stage interprets the instruction and reads necessary registers. Execution, Memory access, and Writeback stages perform ALU operations, handle data memory, and update

register values respectively, with dedicated modules handling each function in parallel for improved performance.

5 Fetch Cycle

The Fetch Cycle consists of the following modules:

- Program Counter
- Instruction Memory
- Multiplexer
- Adder

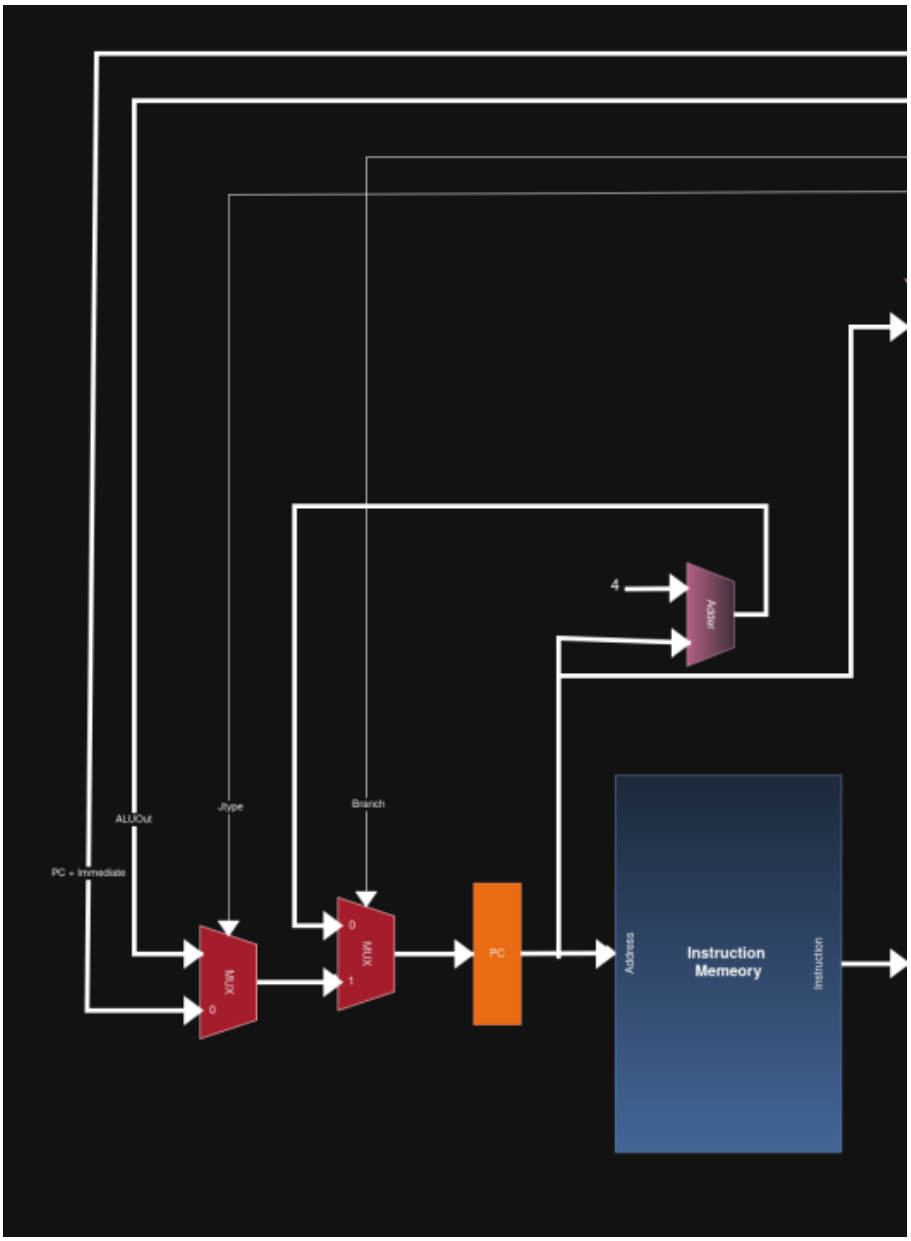


Figure 2: Fetch Stage Block Diagram

5.1 Program Counter

The Program Counter (PC) holds the address of the current instruction being fetched from memory. It ensures the processor executes instructions in the correct sequence. After each instruction, the PC is updated to point to the next instruction, or to a new address in case of branches or jumps.

5.1.1 Design Choices

1. **Synchronous Update with Clock:** The `PC_out` is updated on the positive edge of the clock, which aligns with typical synchronous design principles used in pipeline architectures.

2. **Asynchronous Reset:** The reset signal is handled asynchronously. This allows the PC to reset immediately, regardless of the clock, which is useful for global initialization.

3. **Simple and Minimal Design:** The module has only essential functionality: receive a new PC input and output it on the next clock cycle, or reset to 0.

4. **32-bit Address Width:** The PC is 32 bits wide, which is standard for RISC-V RV32-based designs.

5.1.2 Limitations

1. **No Stall or Freeze Capability:** There is no stall signal to hold the PC value, critical in pipelined processors to handle control hazards.

2. **No Branch/Jump Handling Logic:** The PC simply takes the input as-is; branch/jump target logic must exist externally.

3. **No Instruction Memory Boundary Checking.**

4. **No Support for Delayed PC Update.**

5. **Asynchronous Reset May Lead to Glitches** in some FPGAs or ASICs if not synchronized properly.

5.1.3 Sample Waveform

Figure 3: Program Counter Sample Waveform

5.2 Instruction Memory

Instruction Memory stores the program's machine code instructions that the processor needs to execute. During the Fetch stage, the instruction at the address specified by the

Program Counter is read from this memory. It is typically read-only and separate from data memory to simplify access.

5.2.1 Design Choices

Word-Aligned Addressing (`read_address[7:2]`) Used bits [7:2] to index 64 words (4-byte aligned instructions), enabling access to 256 bytes of instruction memory. Ensures proper word-aligned addressing for 32-bit instructions in RISC-V.

Instruction Memory Size (`I_Mem[0:63]`) Memory is statically defined to hold 64 instructions, sufficient for small to mid-sized programs or testbenches.

Hex File Initialization (`$readmemh`) The module loads instructions from an external hex file at simulation start, simplifying testing and decoupling the processor logic from the program content.

Negative Edge Clocking (`@(negedge clk)`) Instructions are fetched on the falling edge of the clock to synchronize with a positive-edge sampling Fetch/Decode register.

5.2.2 Limitations

1. **Fixed Size:** Only 64 instructions maximum.
2. **No Write Support:** Read-only after initialization.
3. **No Byte-Addressing.**
4. **Lack of Timing Control:** Does not model realistic memory latency.
5. **No Support for Caching or Memory Hierarchy.**
6. **No Error Handling or Protection.**
7. **Not Synthesizable (in current form):** `$readmemh` is simulation-only.

5.2.3 Sample Waveform

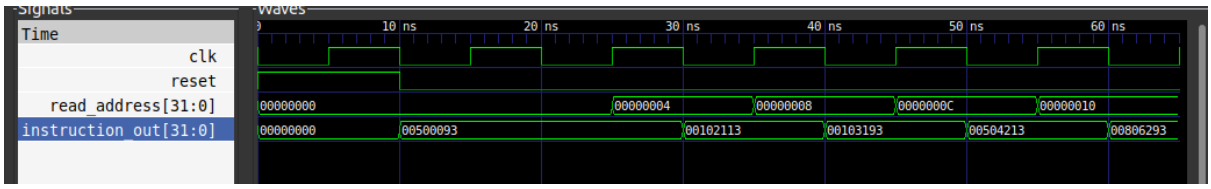


Figure 4: Instruction Memory Sample Waveform

5.3 Use of Two Multiplexers in Fetch Stage

To select between `PC + 4` (default next instruction), branch target address, and jump target address (e.g., JAL, JALR).

5.3.1 Design Choices

1. **Combinational Logic:** Implemented using a simple `assign` statement – purely combinational with zero clock latency.
2. **2-to-1 Multiplexer:** Chooses between two 32-bit inputs A and B based on a 1-bit control signal `select`.
3. **Compact and Readable Syntax:** The ternary operator (`?  :`) minimizes code complexity.

5.3.2 Limitations

1. **No Timing or Enable Control:** The module does not support any clock or enable signal.
2. **No Default or Undefined Handling:** If `select` is driven by `'x'`, the output may also become undefined.

5.4 32-bit Adder Module

Usage in Fetch Stage: Increment the program counter to point to the next instruction (default behavior when there is no branch or jump).

5.4.1 Design Choices

- **Combinational Logic:** Produces immediate outputs in response to changes in input operands.
- **Fixed 32-bit Operand Width:** Aligns with RISC-V RV32I standard.
- **Simplicity and Focus:** No flags, carry-out bits, or clocking mechanisms.
- **Stateless Operation:** No internal state, clocks, or resets.

5.4.2 Limitations

- No Signed Arithmetic Support.
- No Overflow or Carry Detection.
- Fixed Bit Width – not scalable without manual modification.
- No Timing or Delay Considerations.

5.4.3 Sample Waveform

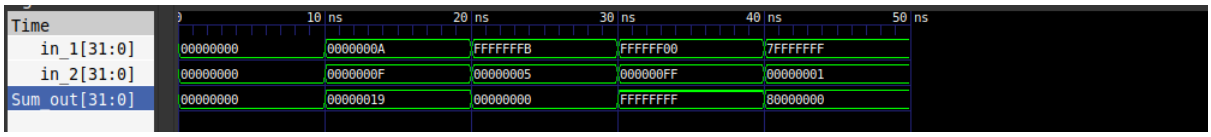


Figure 5: 32-bit Adder Sample Waveform

6 Decode Stage

The Decode stage of the pipelined RISC-V processor is responsible for interpreting instructions and preparing operands and control signals for the Execute stage. It consists of three main components: the Control Unit, Register File, and Immediate Generator.

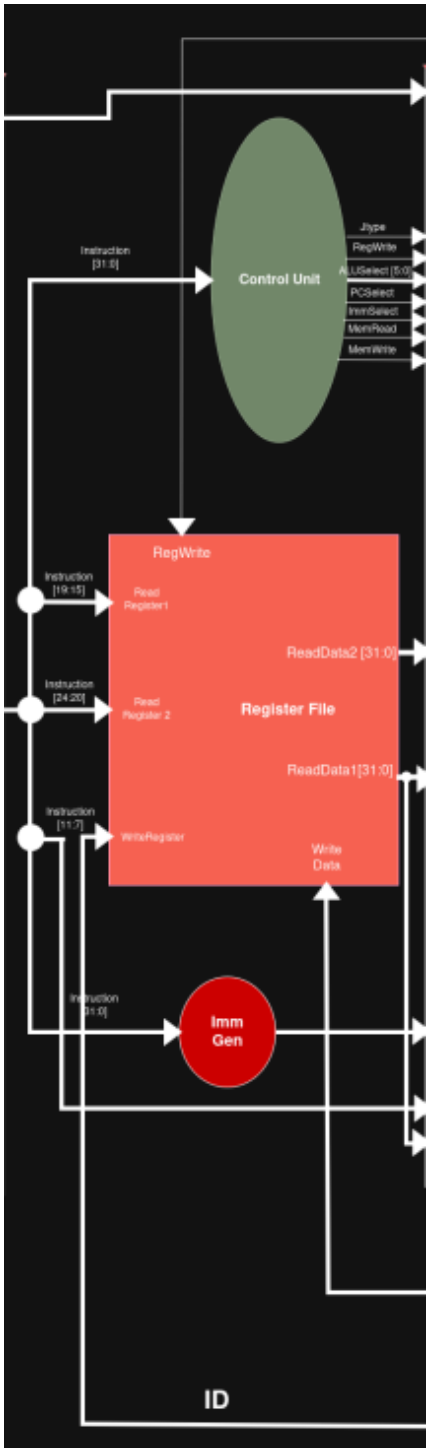


Figure 6: Decode Stage Block Diagram

6.1 Control Unit

1. **Purpose:** Decodes the 32-bit RISC-V instruction and generates all necessary control signals for the Execute, Memory, and Writeback stages.
2. **Inputs:** `instruction[31:0]`: The full 32-bit instruction.
3. **Outputs:**
   - `aluSelect[5:0]`: Specifies the ALU operation.
   - `MemWrite`: Enables write to data memory.
   - `MemRead`: Enables read from data memory.
   - `ImmSelect`: Enables selection of immediate operands.
   - `PCSelect`: Controls PC source selection.
   - `regWrite`: Enables writing to the register file.
   - `Jtype`: Indicates jump-type instruction (JAL/JALR).
4. **Opcode Decoding:** The module decodes the 7-bit opcode to determine the instruction type. Additional fields `funct3` and `funct7` are used for further classification.
5. **ALU Operation Encoding (`aluSelect`):** A 6-bit custom ALU operation code is assigned for each instruction, covering:
   - Arithmetic (ADD, SUB, MUL, etc.)
   - Logical (AND, OR, XOR, shifts)
   - Comparison (SLT, SLTU)
   - Branch conditions (BEQ, BNE, etc.)
   - Load/store indexing
6. **Control Signal Generation:**
   - R-type: sets `aluSelect`, enables `regWrite`, no immediate or memory access.
   - I-type: sets `ImmSelect`, `regWrite`, possibly `MemRead`.
   - S-type: sets `MemWrite`, `ImmSelect`, and `aluSelect`.
   - B-type: sets `aluSelect` for comparison, sets `PCSelect` via branch logic.
   - J-type (JAL, JALR): sets `regWrite`, `ImmSelect`, `PCSelect`, and asserts `Jtype`.
   - U-type (LUI, AUIPC): sets `ImmSelect`, `aluSelect`, `regWrite`.
7. **Special Instructions:** NOP is treated as ADDI to x0 with 0 immediate. FENCE, ECALL, EBREAK are treated as no-ops.
8. **Default Behavior:** All outputs reset to zero at the start of each evaluation to avoid unintended control signal activation.
9. **Combinational Logic:** Implemented using `always @(*)`.

6.1.1 Design Choices

1. Instruction Decoding Based on Opcode The instruction word is partitioned based on the opcode, and in some cases `funct3` and `funct7`, to determine the instruction type.

2. Centralized ALU Control An `aluSelect` signal (6-bit wide) drives the ALU operation, supporting both RV32I and RV32M.

3. Support for RV32M Extension R-type instructions are further differentiated using `funct7` to support multiply/divide instructions.

4. Minimalist Signal Set Only essential control signals are generated: `MemRead`/`MemWrite`, `ImmSelect`, `PCSelect`, `Jtype`, `regWrite`.

5. Combinational Logic Design The Control Unit is implemented as pure combinational logic.

6. Pattern Matching Uses case statements or opcode pattern matching for clean decoding logic.

6.1.2 Limitations

1. **Instruction Set Dependency:** Hardcoded to recognize only RV32I/M. Poor scalability for new extensions.
2. **Limited Control Signal Granularity.**
3. **Lack of Microprogramming or Reconfigurability.**
4. **No Support for Complex Instruction Behaviors** (multicycle operations, etc.)
5. **Hazard Detection is External.**
6. **Lack of Exception and Interrupt Handling.**
7. **No Support for Dynamic Branch Prediction.**
8. **Synchronous Behavior Assumption.**

6.1.3 Sample Waveform

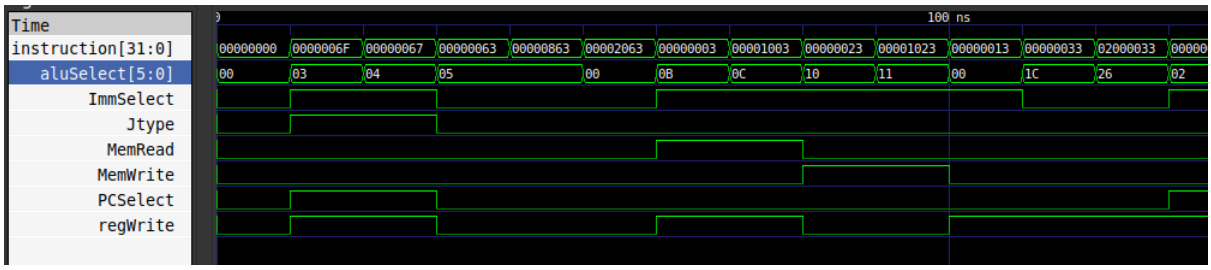


Figure 7: Control Unit Sample Waveform

6.2 Register Files

1. **Purpose:** Stores and provides fast access to the 32 general-purpose RISC-V registers. Supports two simultaneous reads and one write per clock cycle.
2. **Inputs:** `clk`, `reset`, `RegWrite`, `Rs1`, `Rs2`, `Rd`, `Write_data`.
3. **Outputs:** `read_data1` (from Rs1), `read_data2` (from Rs2).
4. **Register Array:** 32 registers, each 32 bits wide: `reg_array[31:0]`.
5. **Write Logic:** Synchronous on positive clock edge. Reset clears all registers. Writes to x0 are ignored.
6. **Read Logic:** Fully combinational. If source register is x0, output is always 0.
7. **RISC-V Compliance:** Enforces x0 = 0. Supports dual-port reads and single-port writes.

6.2.1 Design Choices

1. 32 General-Purpose Registers Aligns with the RISC-V RV32I standard.

2. Synchronous Write Operations Writes occur on the rising edge of the clock with `RegWrite` active and `Rd != 0`.

3. Asynchronous Read Operations Read data becomes available immediately after selecting the source registers.

4. Enforced Zero Register x0 is hardwired to zero. Writes to x0 are ignored; reads always return 0.

5. Reset Behavior On reset, all registers are cleared to zero.

6.2.2 Limitations

1. No Forwarding Support.
2. Glitch Risks from Asynchronous Reads.
3. Single Write Port.
4. No Built-In Debug or Trace Capability.
5. Resource Utilization may be significant on some targets.

6.2.3 Sample Waveform

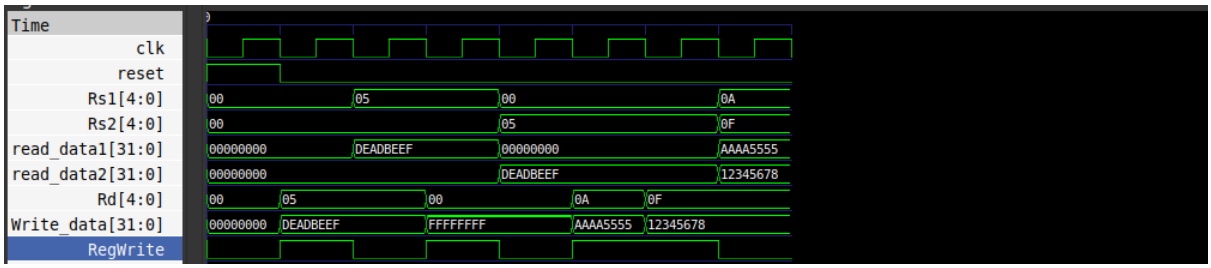


Figure 8: Register File Sample Waveform

6.3 Immediate Generator

The Immediate Generator module extracts the immediate operand from the instruction based on its format and sign-extends it to 32 bits.

6.3.1 I-type Immediate

- **Instructions:** ADDI, SLTI, XORI, Load (LB, LW, etc.), JALR.
- **Extraction:** Bits [31:20].
- **Sign Extension:** `instr[31]` replicated 20 times.

6.3.2 S-type Immediate

- **Instructions:** SB, SH, SW.
- **Extraction:** {`instr[31:25], instr[11:7]`} – 12-bit concatenation.
- **Sign Extension:** `instr[31]` extended into upper 20 bits.

6.3.3 B-type Immediate

- **Instructions:** BEQ, BNE, BLT, BGE, etc.
- **Extraction:** {`instr[31], instr[7], instr[30:25], instr[11:8], 1'b0`} – 13-bit.
- **Sign Extension:** `instr[31]` extended into upper 19 bits.

6.3.4 U-type Immediate

- **Instructions:** LUI, AUIPC.
- **Extraction:** Bits [31:12] directly; lower 12 bits set to zero.

6.3.5 J-type Immediate

- **Instructions:** JAL.
- **Extraction:** {`instr[31], instr[19:12], instr[20], instr[30:21], 1'b0`} – 21-bit.
- **Sign Extension:** `instr[31]` extended into upper 11 bits.

6.3.6 Default Case

If the opcode does not match any known immediate type, the output defaults to zero.

6.3.7 Design Choices

- Format Identification via Opcode (`instr[6:0]`).
- Format-Specific Immediate Extraction as described above.
- Combinational Always Block (`always @(*)`).
- Default Case Handling for unrecognized opcodes.

6.3.8 Limitations

- Lack of Detailed Sub-Decoding (e.g., shift amount for SLLI/SRLI/SRAI).
- No Error Reporting for Invalid Opcodes.
- Lack of Modularity – all logic inline in a single case block.

6.3.9 Sample Waveform

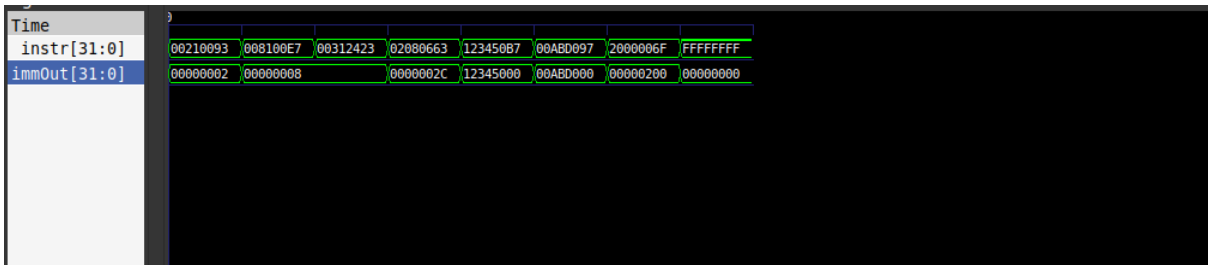


Figure 9: Immediate Generator Sample Waveform

7 Execution Stage

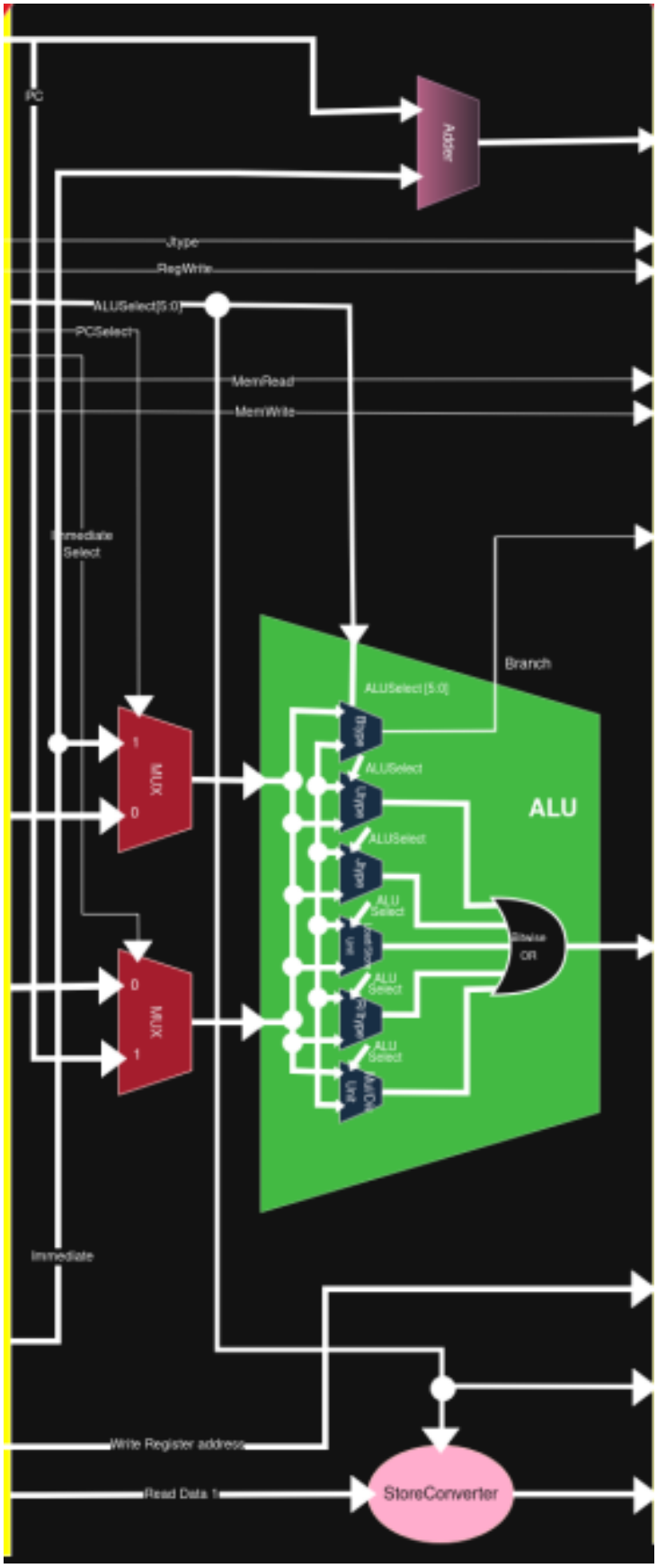


Figure 10: Execution Stage Block Diagram

7.1 Design Overview

1. **Operand Selection via Muxes:**
   - Mux1 selects between PC (`PCD`) and register value (`RegOut1D`) based on `PCSelectD`.
   - Mux2 selects between immediate (`ImmGenOutD`) and register value (`RegOut2D`) based on `ImmSelectD`.
2. **Effective Address Calculation:** Adder computes `PC + Imm` → `PCPlusImmM`.
3. **ALU Operation:** Takes operands from mux outputs and performs the operation specified by `ALUSelectD`. Outputs `BranchM`.
4. **Store Data Processing:** `StoreConverter` prepares store data for memory access.
5. **Pipeline Signal Propagation:** Control and data signals forwarded to the MEM stage unchanged.

7.2 ALU (Arithmetic & Logical Unit)

7.2.1 ALU Design Summary

1. **Purpose:** Performs all computation: arithmetic (add, sub, mul, div), logical (and, or, xor), memory address calculation, jump target generation, and branch condition checking.
2. **Inputs/Outputs:**
   - Inputs: `rs1` (32-bit), `rs2` (32-bit), `aluSelect` (6-bit).
   - Outputs: `result` (32-bit), `branch_taken` (1-bit).
3. **Functional Submodules:** Utype, RItype, muldiv, LoadStoreUnit, Jtype, Btype.
4. **Operation Selection:** Each submodule always active; final result is OR of all submodule outputs. Only one submodule produces a meaningful result at any time.
5. **Design Limitation:** Parallel evaluation may lead to slightly higher power usage.

7.3 Btype – Design Summary

The Btype module evaluates conditional branch instructions and sets the `branch_taken` signal.

Instruction	ALU Select Code	Condition Checked	Description
BEQ	`6'b000101`	rs1 == rs2	Branch if equal
BNE	`6'b000110`	rs1 != rs2	Branch if not equal
BLT	`6'b000111`	$signed(rs1) ¡ $signed(rs2)	Branch if less than
BGE	`6'b001000`	$signed(rs1) ¿= $signed(rs2)	Branch if greater/equal
BLTU	`6'b001001`	rs1 ¡ rs2	Branch if less (unsigned)
BGEU	`6'b001010`	rs1 ¿= rs2	Branch if ge (unsigned)
JAL	`6'b000011`	Unconditional	Treated as always taken
JALR	`6'b000100`	Unconditional	Treated as always taken

Table 7: Btype Supported Instructions

7.4 Jtype – Design Summary

Computes the next PC for JAL and JALR instructions.

- **JAL** (`aluSelect = 6'b000011`): `next_pc = pc + imm`
- **JALR** (`aluSelect = 6'b000100`): `next_pc = (rs1 + imm) & 32'hFFFFFFFE`
- **Default:** `next_pc = 0`

7.5 LoadStoreUnit – Design Summary

Computes the effective memory address: `address = rs1 + imm`.

Instruction	aluSelect	Description
LB	`6'b001011`	Load byte
LH	`6'b001100`	Load halfword
LW	`6'b001101`	Load word
LBU	`6'b001110`	Load byte unsigned
LHU	`6'b001111`	Load halfword unsigned
SB	`6'b010000`	Store byte
SH	`6'b010001`	Store halfword
SW	`6'b010010`	Store word

Table 8: LoadStoreUnit Supported Instructions

7.6 muldiv – Design Summary

Implements RV32M integer multiplication and division.

Instruction	ALU Select	Operation Description
MUL	`6'b100110`	Lower 32 bits of signed multiplication
MULH	`6'b100111`	Upper 32 bits of signed × signed
MULHSU	`6'b101000`	Upper 32 bits of signed × unsigned
MULHU	`6'b101001`	Upper 32 bits of unsigned × unsigned
DIV	`6'b101010`	Signed division (quotient)
DIVU	`6'b101011`	Unsigned division (quotient)
REM	`6'b101100`	Signed remainder
REMU	`6'b101101`	Unsigned remainder

Table 9: muldiv Supported Instructions

Division by zero returns 0. Intermediate products extended to 64 bits. No handling for overflow edge cases (e.g., `INT_MIN / -1`).

7.7 RItype: ALU Logic for R-type and I-type Instructions

Type	Instruction	Operation Description	ALU Control
I-type	ADDI	Addition with immediate	010011
I-type	ANDI	Bitwise AND with immediate	011000
I-type	ORI	Bitwise OR with immediate	010111
I-type	XORI	Bitwise XOR with immediate	010110
I-type	SLLI	Logical left shift	011001
I-type	SRLI	Logical right shift	011010
I-type	SRAI	Arithmetic right shift	011011
I-type	SLTI	Signed less-than comparison	010100
I-type	SLTIU	Unsigned less-than comparison	010101
R-type	ADD	Register addition	011100
R-type	SUB	Register subtraction	100100
R-type	AND	Bitwise AND	100011
R-type	OR	Bitwise OR	100010
R-type	XOR	Bitwise XOR	100000
R-type	SLL	Logical left shift	011101
R-type	SRL	Logical right shift	100001
R-type	SRA	Arithmetic right shift	100101
R-type	SLT	Signed less-than comparison	011110
R-type	SLTU	Unsigned less-than comparison	011111

Table 10: RItype Supported Operations

Shift operations use only `b[4:0]`. Signed operations use Verilog's `$signed()` operator.

7.8 Utype: ALU Logic for U-type Instructions

- **LUI** (`aluSelect = 000001`)**:** Output = `imm_u`
- **AUIPC** (`aluSelect = 000010`)**:** Output = `pc + imm_u`

The immediate value is expected to be pre-shifted by 12 bits externally.

7.9 StoreConverter Module

Performs data masking for store operations to ensure correct data formatting.

Instruction	aluSelect	Action
SB	010000	Retain bits [7:0] via `AND 0x000000FF`
SH	010001	Retain bits [15:0] via `AND 0x0000FFFF`
SW / others	Any other	Pass full 32-bit data unchanged

Table 11: StoreConverter Control Behavior

7.9.1 Sample Waveform

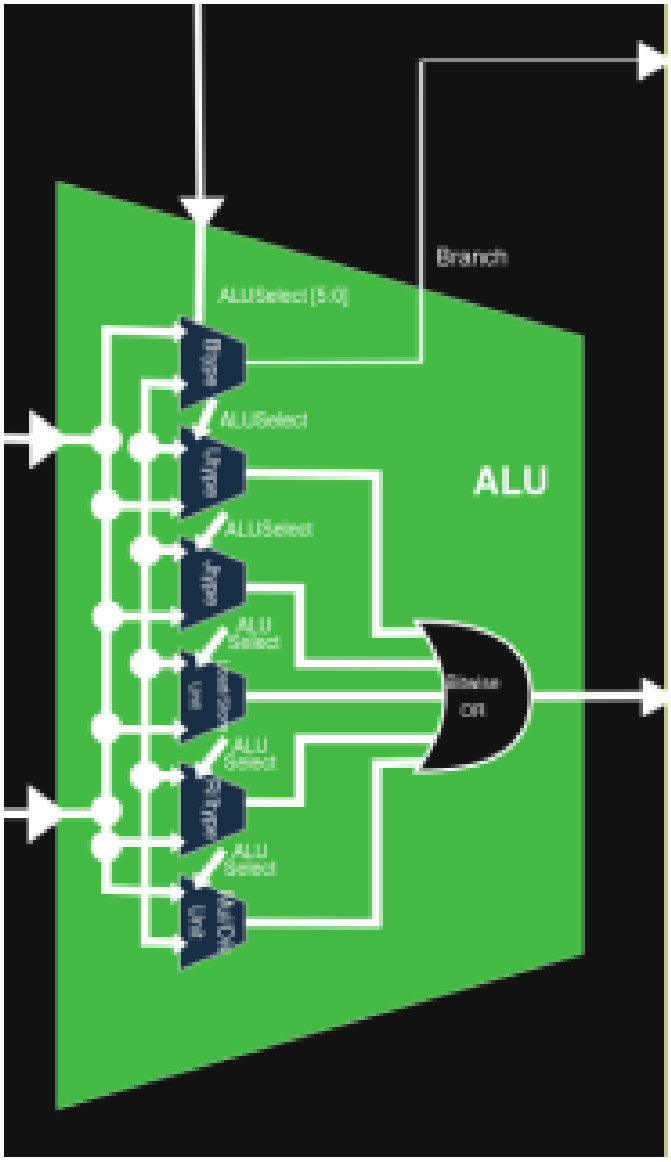


Figure 11: Execution Stage / StoreConverter Sample Waveform

8 Memory Stage

The Memory stage is the fourth stage (IF → ID → EX → MEM → WB). Its core responsibilities include accessing memory for load and store instructions and passing necessary data and control signals to the Writeback stage.

8.1 Data Memory

The Data Memory module is a synchronous memory block supporting load and store operations for a 32-bit RISC-V processor pipeline with 64 words, each 32 bits wide.

8.1.1 Key Features

- **Memory Array:** 64 entries × 32 bits, implemented as a register array.
- **Synchronous Write:** On rising edge of clock when `MemWrite` is asserted.
- **Asynchronous Read:** Immediate output on `MemData_out` when `MemRead` is asserted; zero otherwise.
- **Reset Behavior:** Active reset clears all entries to zero.

8.1.2 Limitations

1. Fixed Memory Size.
2. No Memory-Mapped I/O.
3. No Support for Unaligned Access.
4. No Access Latency Modeling.
5. Lack of Byte/Word Granularity Control.
6. No Bus Interface (AXI, AHB, TileLink).
7. No Exception Handling.
8. No Cache Support.
9. No Memory Protection (MMU).

8.1.3 Sample Waveform

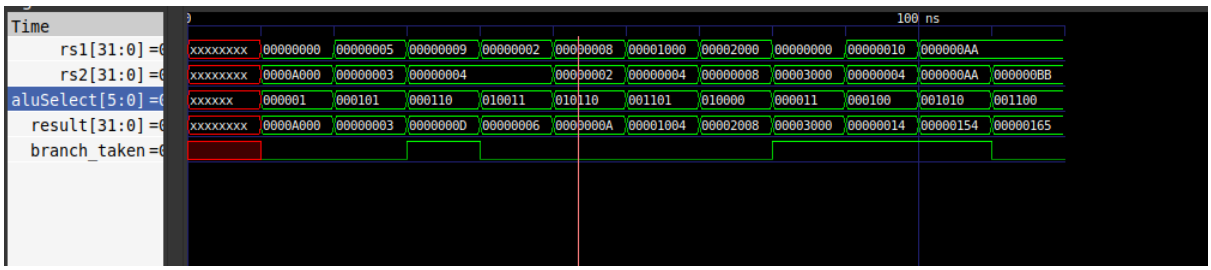


Figure 12: Data Memory Sample Waveform

9 WriteBack Stage

Figure 13: WriteBack Stage Block Diagram

9.1 Design Overview

The WriteBack cycle module selects and prepares data to be written back to the processor's register file.

9.1.1 Key Modules

1. **Mux1:** Selects between `DataMemOutM` (memory data) and `ALUOutM` (ALU result), controlled by `MemReadM`.
2. **Adder:** Adds 4 to the `LoadConverter` output for jump return address (`PC + 4`).
3. **Mux2:** Selects between the incremented address (for jumps) and load/ALU data, controlled by `JtypeM`.

9.2 LoadConverter Module

Processes data loaded from memory before it is written back to the register file.

9.2.1 Data Conversion

- Signed loads (LB, LH): sign-extend the smaller data size.
- Unsigned loads (LBU, LHU): zero-extend the smaller data size.
- Default / LW: pass input data through unchanged.

9.2.2 Design Choices

- Case-Based Decoding via `aluSelect`.
- Simple Combinational Logic (`always @(*)`).
- Sign vs. Zero Extension using predefined masks (`32'hffffff00`, `32'hffff0000`).

9.2.3 Limitations

- Limited to Basic Load Types (LB, LH, LBU, LHU).
- Hardcoded Masks – requires manual changes for other architectures.
- No Error Detection for invalid `aluSelect`.
- No Misalignment Handling.

9.2.4 Sample Waveform

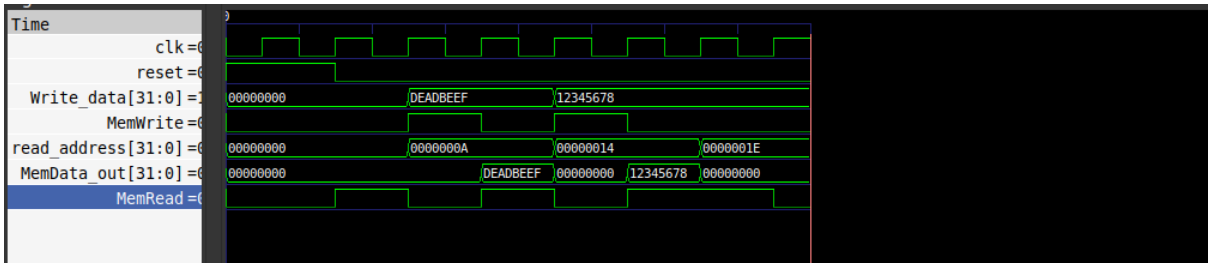


Figure 14: WriteBack Stage Sample Waveform

Part III
Lab Part 3 – Integration

10 Integration of Each Stage

10.1 Instruction Fetch Stage Integration

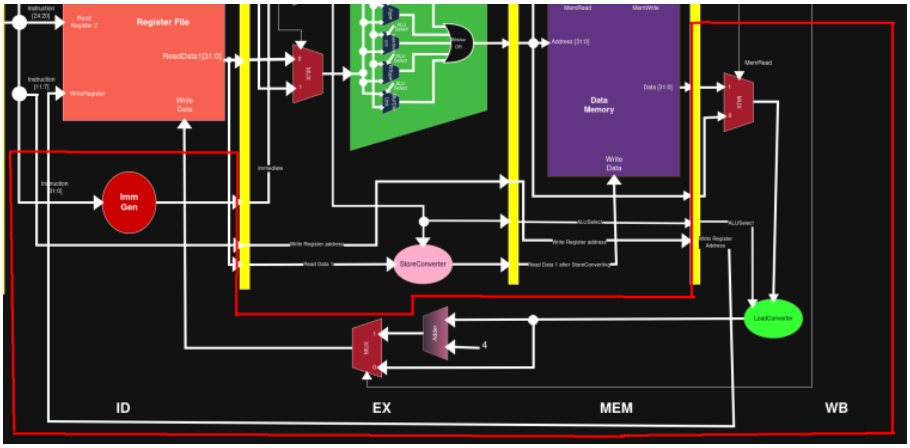


Figure 15: Fetch Stage Integration – Full Datapath

The Fetch stage is the first stage of the pipelined RISC-V processor. It is responsible for computing the next PC, fetching the instruction from instruction memory, and passing the PC and instruction to the Decode stage.

10.1.1 Functional Overview

1. Program Counter (PC) Logic

- Two 2-to-1 multiplexers choose the correct next PC:
  - `JtypeEMux`: selects between `ALUOutM` (JALR target) and `PCPlusImmM` (JAL target).
  - `BranchEMux`: selects between jump target and `PC + 4` depending on `BranchM`.

2. PC + 4 Calculation Adder computes `PC + 4` for sequential execution.

3. Instruction Memory Access `Instruction_Mem` takes the current PC and outputs the 32-bit instruction.

4. Outputs to Decode Stage `PCD` (current PC) and `InstrD` (current instruction) forwarded via the IF/ID pipeline register.

10.1.2 Design Choices

- Two-stage muxing for clean modular handling of jumps and branches.
- Branch/jump control signals come from the Memory stage.
- Adder and `Instruction_Mem` separated for modularity.

10.1.3 Limitations

- No stall or flush logic.
- Instruction memory assumed to read instantly.
- No instruction cache (I-Cache).
- Jumps and branches resolved late (MEM stage) – several cycles of delay.

10.2 Instruction Decode (ID) Stage – Integration

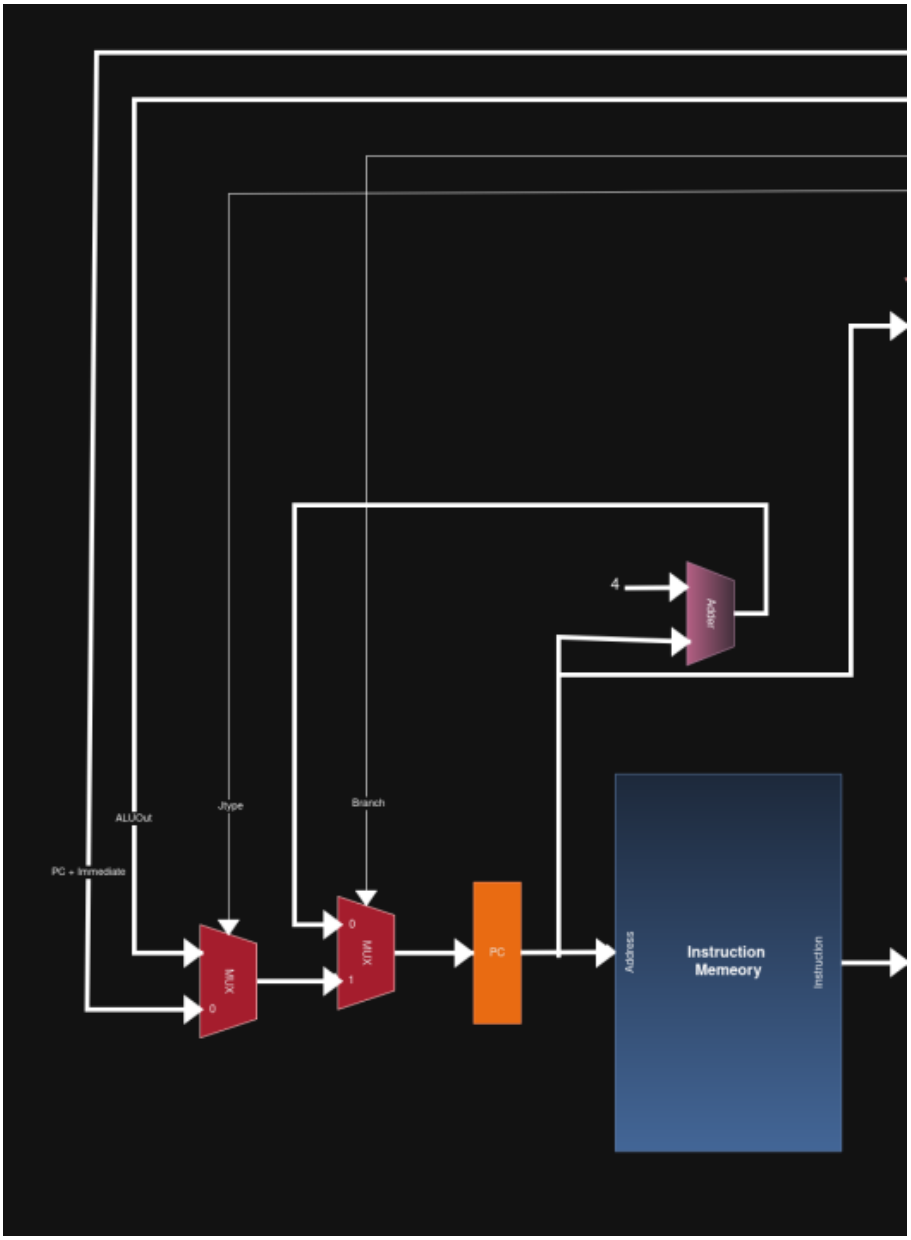


Figure 16: Decode Stage Integration – Full Datapath

The `decode_cycle` module decodes the fetched instruction, reads source registers, generates the immediate value, and prepares control signals.

10.2.1 Submodule Integration

1. Control Unit (ControlUnitD) Inputs: `instructionF`. Outputs: `aluSelect`, `MemWrite`, `MemRead`, `ImmSelect`, `PCSelect`, `RegWrite`, `Jtype`.

2. Register File (Reg_File_D) Rs1 = `instructionF[19:15]`, Rs2 = `instructionF[24:20]`. Reads from register file; writes from WB stage via `WriteAddressW`, `writeDataW`, `RegWriteW`.

3. Immediate Generator (ImmGen_D) Produces sign-extended immediate `ImmGenOutE` based on full instruction.

4. Output Assignments `WriteAddressE = instructionF[11:7]`, `PCE = PCF`.

10.2.2 Limitations

1. No hazard detection or stalling logic.
2. No immediate multiplexer.
3. No forwarding logic.
4. Write-back only on positive edge.

10.3 Execution Stage Integration

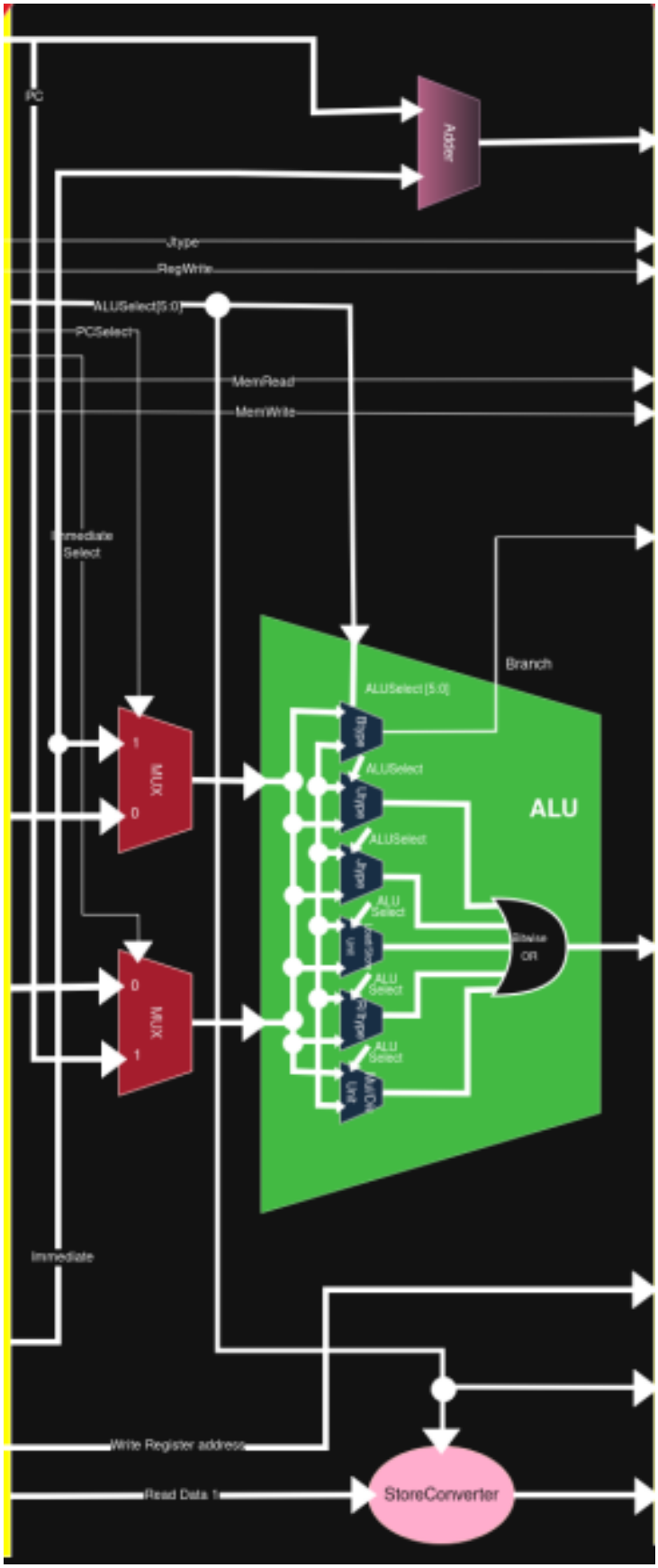


Figure 17: Execution Stage Integration – Full Datapath

10.3.1 Design Objectives

Perform ALU operations, support immediate and register-based operations, evaluate branch conditions, prepare memory write data, and propagate signals to the MEM stage.

10.3.2 Integrated Modules

- **Mux1 and Mux2:** Select operands for the ALU based on `PCSelectD` and `ImmSelectD`.
- **ALU:** Outputs `ALUOutM` and `BranchM`.
- **Adder:** Computes branch/jump target as `PCD + ImmGenOutD`.
- **StoreConverter:** Pre-formats data for memory write.

10.3.3 Design Choices

Modularization, clean separation of ALU and branch logic, forwarding readiness, pipeline-friendly combinational paths.

10.3.4 Limitations

No hazard mitigation, limited ALU scope, no bypass support for StoreConverter.

10.4 Integration of Memory Stage

The Memory stage acts as a bridge between Execute and WriteBack stages, communicates with Data Memory, and supplies branch/jump data to the Fetch stage.

10.4.1 Key Signals Passed

- **From Execute:** ALU result (memory address), store data, control signals (MemRead, MemWrite, Branch, Jtype, RegWrite).
- **To WriteBack:** Memory output data, ALU result, write-back destination register, control signals.
- **To Fetch:** Branch target address, branch and jump indicators.

10.4.2 Limitations

No internal hazard detection, late branch resolution increases misprediction penalty, single-cycle memory assumption, no exception handling.

10.5 Integration Summary of the WriteBack Stage

10.5.1 Key Inputs (from MEM/WB register)

`DataMemOutM`, `ALUOutM`, `ALUSelectM`, `JtypeM`, `MemReadM`, `RegWriteM`, `WriteAddressM`.

10.5.2 Internal Units

- Mux1 → LoadConverter → Mux2 → `RegInDataD`
- Adder computes `PC + 4` for jump return address.

10.5.3 Key Outputs

`RegInDataD` (final write data), `WriteAddressD` (destination register), `RegWriteD` (write enable).

11 Design Overview of Inter-Stage Pipeline Registers

Inter-stage pipeline registers hold and transfer data and control signals between pipeline stages on each clock cycle.

Pipeline Register	Connects	Purpose
IF/ID	Fetch → Decode	Stores fetched instruction and PC. Enables decoding in the next cycle.
ID/EX	Decode → Execute	Holds decoded control signals, operand values, immediate values, and instruction fields.
EX/MEM	Execute → Memory	Transfers ALU result, branch/jump decisions, memory operation flags, and destination register info.
MEM/WB	Memory → WriteBack	Carries memory or ALU results, register write-back info, and jump data to the register file.

Table 12: Inter-Stage Pipeline Registers

11.0.1 Design Choices

Granular fields, signal grouping, synchronous clocked behavior with `clk` and `reset`, flush and stall support via zeroing/bypassing.

11.0.2 Limitations

Fixed-width data buses, no ECC or error detection, no internal forwarding.

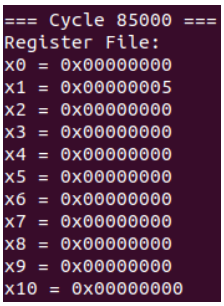
12 Testing for Instructions after Overall Integration

12.1 I-type Instructions

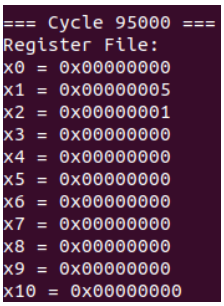
```
00500093      // ADDI    x1, x0, 5
00102113      // SLTI    x2, x0, 1
00103193      // SLTIU   x3, x0, 1
00504213      // XORI    x4, x0, 5
00806293      // ORI     x5, x0, 8
0040F313      // ANDI    x6, x1, 4
00109393      // SLLI    x7, x1, 1
0010D413      // SRLI    x8, x1, 1
4010D493      // SRAI    x9, x1, 1
```

After 85000 ps (85 ns), these instructions start storing data on registers (after completing the pipeline).

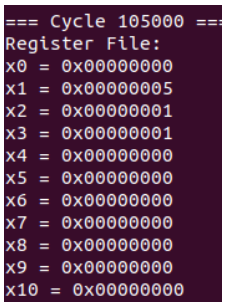
Figure 19: I-type Instructions – Waveform Overview



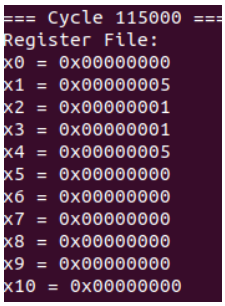
(a) ADDI x1



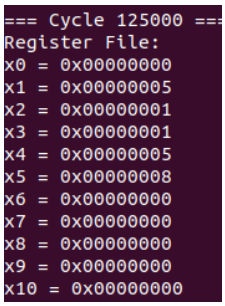
(b) SLTI x2



(c) SLTIU x3

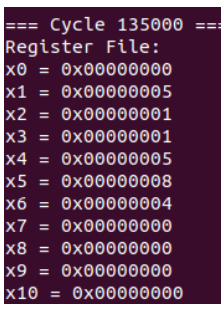


(d) XORI x4

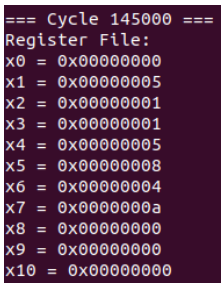


(e) ORI x5

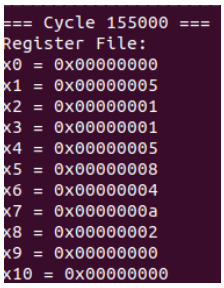
Figure 20: I-type Instruction Register Results (Part 1)



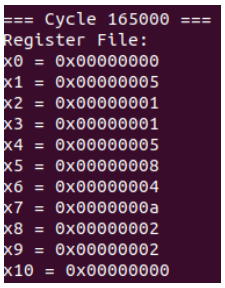
(a) ANDI x6



(b) SLLI x7



(c) SRLI x8



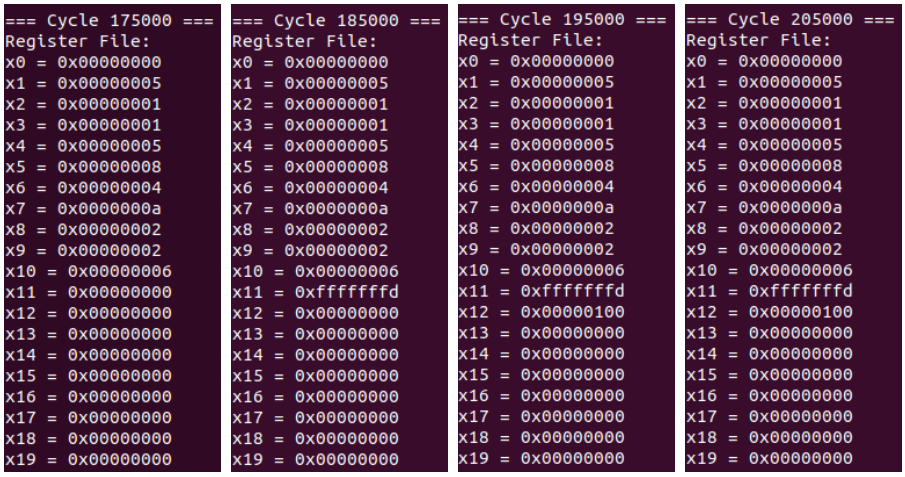
(d) SRAI x9

Figure 21: I-type Instruction Register Results (Part 2)

12.2 R-type Instructions

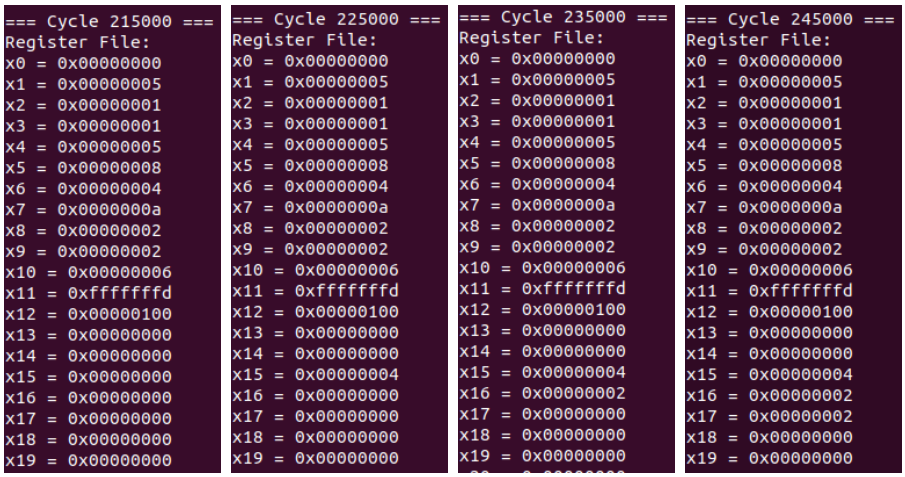
```
00208533        // ADD  x10, x1, x2
405205b3        // SUB  x11, x4, x5
00129633        // SLL  x12, x5, x1
0020a6b3        // SLT  x13, x1, x2
0020b733        // SLTU x14, x1, x2
0020c7b3        // XOR  x15, x1, x2
0020d833        // SRL  x16, x1, x2
4020d8b3        // SRA  x17, x1, x2
0020e933        // OR   x18, x1, x2
0020f9b3        // AND  x19, x1, x2
```

By the end of 265 ns, these R-type instructions fill the registers with calculated values.



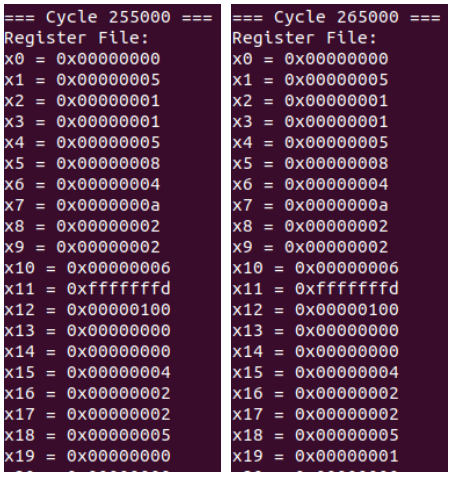
(a) ADD x10 (b) SUB x11 (c) SLL x12 (d) SLT x13

Figure 22: R-type Instruction Register Results (Part 1)



(a) SLTU x14 (b) XOR x15 (c) SRL x16 (d) SRA x17

Figure 23: R-type Instruction Register Results (Part 2)



(a) OR x18 (b) AND x19

Figure 24: R-type Instruction Register Results (Part 3)

12.3 Multiplication and Division Instructions

```
02508533      // MUL    x10, x1, x5
025095B3      // MULH   x11, x1, x5
0250A633      // MULHSU x12, x1, x5
0250B6B3      // MULHU  x13, x1, x5
0212C733      // DIV    x14, x5, x1
0212D7B3      // DIVU   x15, x5, x1
0212E833      // REM    x16, x5, x1
0212f8B3      // REMU   x17, x5, x1
```

```
=== Cycle 335000 ===
Register File:
x0 = 0x00000000
x1 = 0x00000005
x2 = 0x00000001
x3 = 0x00000001
x4 = 0x00000005
x5 = 0x00000008
x6 = 0x00000004
x7 = 0x0000000a
x8 = 0x00000002
x9 = 0x00000002
x10 = 0x00000028
x11 = 0x00000000
x12 = 0x00000000
x13 = 0x00000000
x14 = 0x00000001
x15 = 0x00000001
x16 = 0x00000003
x17 = 0x00000003
x18 = 0x00000005
x19 = 0x00000001
x20 = 0x00000000
x21 = 0x00000000
x22 = 0x00000000
x23 = 0x00000000
x24 = 0x00000000
x25 = 0x00000000
x26 = 0x00000000
x27 = 0x00000000
x28 = 0x00000000
x29 = 0x00000000
x30 = 0x00000000
x31 = 0x00000000
Data Memory:
Mem[0] = 0x00000000
Mem[1] = 0x00000000
Mem[2] = 0x00000000
Mem[3] = 0x00000000
Mem[4] = 0x00000000
Mem[5] = 0x00000000
Mem[6] = 0x00000000
Mem[7] = 0x0000000a
Mem[8] = 0x00000000
Mem[9] = 0x00000000
Mem[10] = 0x00000000
```

(a) MUL x10

```
=== Cycle 345000 ===
Register File:
x0 = 0x00000000
x1 = 0x00000005
x2 = 0x00000001
x3 = 0x00000001
x4 = 0x00000005
x5 = 0x00000008
x6 = 0x00000004
x7 = 0x0000000a
x8 = 0x00000002
x9 = 0x00000002
x10 = 0x00000028
x11 = 0x00000000
x12 = 0x00000000
x13 = 0x00000000
x14 = 0x00000001
x15 = 0x00000001
x16 = 0x00000003
x17 = 0x00000003
x18 = 0x00000005
x19 = 0x00000001
x20 = 0x00000000
x21 = 0x00000000
x22 = 0x00000000
x23 = 0x00000000
x24 = 0x00000000
x25 = 0x00000000
x26 = 0x00000000
x27 = 0x00000000
x28 = 0x00000000
x29 = 0x00000000
x30 = 0x00000000
x31 = 0x00000000
Data Memory:
Mem[0] = 0x00000000
Mem[1] = 0x00000000
Mem[2] = 0x00000000
Mem[3] = 0x00000000
Mem[4] = 0x00000000
Mem[5] = 0x00000000
Mem[6] = 0x00000000
Mem[7] = 0x0000000a
Mem[8] = 0x0000000a
Mem[9] = 0x00000000
Mem[10] = 0x00000000
```

(b) MULH x11 x12

```
=== Cycle 355000 ===
Register File:
x0 = 0x00000000
x1 = 0x00000005
x2 = 0x00000001
x3 = 0x00000001
x4 = 0x00000005
x5 = 0x00000008
x6 = 0x00000004
x7 = 0x0000000a
x8 = 0x00000002
x9 = 0x00000002
x10 = 0x00000028
x11 = 0x00000000
x12 = 0x00000000
x13 = 0x00000000
x14 = 0x00000001
x15 = 0x00000001
x16 = 0x00000003
x17 = 0x00000003
x18 = 0x00000005
x19 = 0x00000001
x20 = 0x00000000
x21 = 0x00000000
x22 = 0x00000000
x23 = 0x00000000
x24 = 0x00000000
x25 = 0x00000000
x26 = 0x00000000
x27 = 0x00000000
x28 = 0x00000000
x29 = 0x00000000
x30 = 0x00000000
x31 = 0x00000000
Data Memory:
Mem[0] = 0x00000000
Mem[1] = 0x00000000
Mem[2] = 0x00000000
Mem[3] = 0x00000000
Mem[4] = 0x00000000
Mem[5] = 0x00000000
Mem[6] = 0x00000000
Mem[7] = 0x0000000a
Mem[8] = 0x0000000a
Mem[9] = 0x0000000a
Mem[10] = 0x00000000
```

(c) MULHSU

Figure 25: Multiply Instruction Register Results (Part 1)

```
=== Cycle 375000 ===
Register File:
x0 = 0x00000000
x1 = 0x00000005
x2 = 0x00000001
x3 = 0x00000001
x4 = 0x00000005
x5 = 0x00000008
x6 = 0x00000004
x7 = 0x0000000a
x8 = 0x00000002
x9 = 0x00000002
x10 = 0x00000028
x11 = 0x00000000
x12 = 0x00000000
x13 = 0x00000000
x14 = 0x00000001
x15 = 0x00000001
x16 = 0x00000003
x17 = 0x00000003
x18 = 0x00000005
x19 = 0x00000001
x20 = 0xfffffff0a
x21 = 0x00000000
x22 = 0x00000000
x23 = 0x00000000
x24 = 0x00000000
x25 = 0x00000000
x26 = 0x00000000
x27 = 0x00000000
x28 = 0x00000000
x29 = 0x00000000
x30 = 0x00000000
x31 = 0x00000000
```

(a) DIV x14

```
=== Cycle 385000 ===
Register File:
x0 = 0x00000000
x1 = 0x00000005
x2 = 0x00000001
x3 = 0x00000001
x4 = 0x00000005
x5 = 0x00000008
x6 = 0x00000004
x7 = 0x0000000a
x8 = 0x00000002
x9 = 0x00000002
x10 = 0x00000028
x11 = 0x00000000
x12 = 0x00000000
x13 = 0x00000000
x14 = 0x00000001
x15 = 0x00000001
x16 = 0x00000003
x17 = 0x00000003
x18 = 0x00000005
x19 = 0x00000001
x20 = 0xfffffff0a
x21 = 0xffff000a
x22 = 0x00000000
x23 = 0x00000000
x24 = 0x00000000
x25 = 0x00000000
x26 = 0x00000000
x27 = 0x00000000
x28 = 0x00000000
x29 = 0x00000000
x30 = 0x00000000
x31 = 0x00000000
```

(b) DIVU x15

```
=== Cycle 395000 ===
Register File:
x0 = 0x00000000
x1 = 0x00000005
x2 = 0x00000001
x3 = 0x00000001
x4 = 0x00000005
x5 = 0x00000008
x6 = 0x00000004
x7 = 0x0000000a
x8 = 0x00000002
x9 = 0x00000002
x10 = 0x00000028
x11 = 0x00000000
x12 = 0x00000000
x13 = 0x00000000
x14 = 0x00000001
x15 = 0x00000001
x16 = 0x00000003
x17 = 0x00000003
x18 = 0x00000005
x19 = 0x00000001
x20 = 0xfffffff0a
x21 = 0xffff000a
x22 = 0x0000000a
x23 = 0x00000000
x24 = 0x00000000
x25 = 0x00000000
x26 = 0x00000000
x27 = 0x00000000
x28 = 0x00000000
x29 = 0x00000000
x30 = 0x00000000
x31 = 0x00000000
```

(c) REM x16

```
=== Cycle 405000 ===
Register File:
x0 = 0x00000000
x1 = 0x00000005
x2 = 0x00000001
x3 = 0x00000001
x4 = 0x00000005
x5 = 0x00000008
x6 = 0x00000004
x7 = 0x0000000a
x8 = 0x00000002
x9 = 0x00000002
x10 = 0x00000028
x11 = 0x00000000
x12 = 0x00000000
x13 = 0x00000000
x14 = 0x00000001
x15 = 0x00000001
x16 = 0x00000003
x17 = 0x00000003
x18 = 0x00000005
x19 = 0x00000001
x20 = 0xfffffff0a
x21 = 0xffff000a
x22 = 0x0000000a
x23 = 0x0000000a
x24 = 0x00000000
x25 = 0x00000000
x26 = 0x00000000
x27 = 0x00000000
x28 = 0x00000000
x29 = 0x00000000
x30 = 0x00000000
x31 = 0x00000000
```

(d) REMU x17

Figure 26: Divide/Remainder Instruction Register Results

12.4 Branch Instructions

```
fe310ee3     // beq  x2, x3, -4
ff079ee3     // bne  x15, x16, -4
fe114ee3     // blt  x2, x1, -4
fe20dee3     // bge  x1, x2, -4
fe116ee3     // bltu x2, x1, -4
fe20fee3     // bgeu x1, x2, -4
```

Each branch instruction is tested individually with the others commented out.

```
=== Cycle 425000 ===
Register File:
x0 = 0x00000000
x1 = 0x00000005
x2 = 0x00000001
x3 = 0x00000001
x4 = 0x00000005
x5 = 0x00000008
x6 = 0x00000004
x7 = 0x0000000a
x8 = 0x00000002
x9 = 0x00000002
x10 = 0x00000028
x11 = 0x00000000
x12 = 0x00000000
x13 = 0x00000000
x14 = 0x00000001
x15 = 0x00000001
x16 = 0x00000003
x17 = 0x00000003
x18 = 0x00000005
x19 = 0x00000001
x20 = 0xffffff0a
x21 = 0xffff000a
x22 = 0x0000000a
x23 = 0x0000000a
x24 = 0x0000000a
x25 = 0x00929000
x26 = 0x00000000
x27 = 0x00000000
x28 = 0x00000000
x29 = 0x00000000
x30 = 0x00000000
x31 = 0x00000000
```

(a) BEQ waveform

```
=== Cycle 435000 ===
Register File:
x0 = 0x00000000
x1 = 0x00000005
x2 = 0x00000001
x3 = 0x00000001
x4 = 0x00000005
x5 = 0x00000008
x6 = 0x00000004
x7 = 0x0000000a
x8 = 0x00000002
x9 = 0x00000002
x10 = 0x00000028
x11 = 0x00000000
x12 = 0x00000000
x13 = 0x00000000
x14 = 0x00000001
x15 = 0x00000001
x16 = 0x00000003
x17 = 0x00000003
x18 = 0x00000005
x19 = 0x00000001
x20 = 0xffffff0a
x21 = 0xffff000a
x22 = 0x0000000a
x23 = 0x0000000a
x24 = 0x0000000a
x25 = 0x00929000
x26 = 0x0000a090
x27 = 0x00000000
x28 = 0x00000000
x29 = 0x00000000
x30 = 0x00000000
x31 = 0x00000000
```

(b) BNE waveform

Figure 27: Branch Instruction Waveforms (Part 1)

```
=== Cycle 445000 ===
Register File:
x0 = 0x00000000
x1 = 0x00000005
x2 = 0x00000001
x3 = 0x00000001
x4 = 0x00000005
x5 = 0x00000008
x6 = 0x00000004
x7 = 0x0000000a
x8 = 0x00000002
x9 = 0x00000002
x10 = 0x00000028
x11 = 0x00000000
x12 = 0x00000000
x13 = 0x00000000
x14 = 0x00000001
x15 = 0x00000001
x16 = 0x00000003
x17 = 0x00000003
x18 = 0x00000005
x19 = 0x00000001
x20 = 0xffffff0a
x21 = 0xffff000a
x22 = 0x0000000a
x23 = 0x0000000a
x24 = 0x0000000a
x25 = 0x00929000
x26 = 0x0000a090
x27 = 0x00000094
x28 = 0x00000000
x29 = 0x00000000
x30 = 0x00000000
x31 = 0x00000000
```

(a) BLT waveform

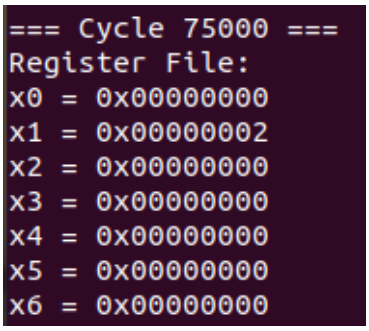
```
=== Cycle 445000 ===
Register File:
x0 = 0x00000000
x1 = 0x00000005
x2 = 0x00000001
x3 = 0x00000001
x4 = 0x00000005
x5 = 0x00000008
x6 = 0x00000004
x7 = 0x0000000a
x8 = 0x00000002
x9 = 0x00000002
x10 = 0x00000028
x11 = 0x00000000
x12 = 0x00000000
x13 = 0x00000000
x14 = 0x00000001
x15 = 0x00000001
x16 = 0x00000003
x17 = 0x00000003
x18 = 0x00000005
x19 = 0x00000001
x20 = 0xffffff0a
x21 = 0xffff000a
x22 = 0x0000000a
x23 = 0x0000000a
x24 = 0x0000000a
x25 = 0x00929000
x26 = 0x0000a090
x27 = 0x00000000
x28 = 0x00000094
x29 = 0x00000000
x30 = 0x00000000
x31 = 0x00000000
```

(b) BGE waveform

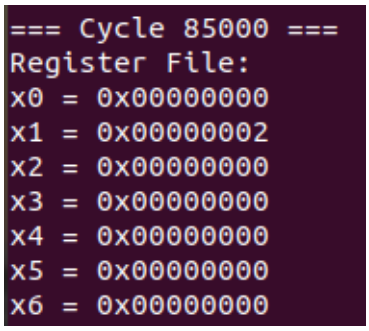
Figure 28: Branch Instruction Waveforms (Part 2)

12.5 Store Instructions

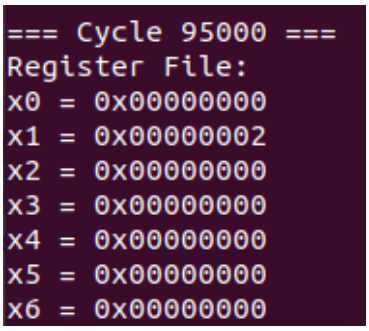
```
0x00708123   // sb x7, 2(x1)
0x007091a3   // sh x7, 3(x1)
0x0070a223   // sw x7, 4(x1)
```



(a) sb x7, 2(x1)



(b) sh x7, 3(x1)

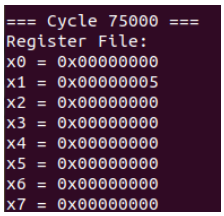


(c) sw x7, 4(x1)

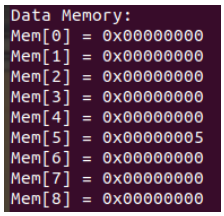
Figure 29: Store Instruction Register Results

12.6 Load Instructions

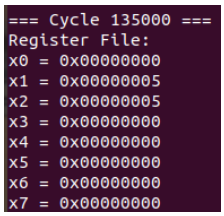
```
0x00208a03   // lb  x20, 2(x1)
0x00209a83   // lh  x21, 2(x1)
0x0020ab03   // lw  x22, 2(x1)
0x0020cb83   // lbu x23, 2(x1)
0x0020dc03   // lhu x24, 2(x1)
```



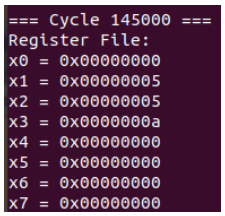
(a) lb x20



(b) lh x21



(c) lw x22



(d) lbu x23

Figure 30: Load Instruction Register Results

12.7 U-type Instructions

```
0x00929cb7   // lui   x25, 2345
0x0000ad17   // auipc x26, 10
```

```
=== Cycle 75000 ===
Register File:
x0 = 0x00000000
x1 = 0x00000002
x2 = 0x00000000
x3 = 0x00000000
x4 = 0x00000000
x5 = 0x00000000
x6 = 0x00000000
```

(a) lui x25, 2345

```
=== Cycle 85000 ===
Register File:
x0 = 0x00000000
x1 = 0x00000002
x2 = 0x00000004
x3 = 0x00000000
x4 = 0x00000000
x5 = 0x00000000
x6 = 0x00000000
```

(b) auipc x26, 10

Figure 31: U-type Instruction Register Results

12.8 Jump Instructions

```
0xffdffdef   // jal x27, -4
0xffc08e67   // jalr x28, -4(x1)  (one tested at a time)
```

```
=== Cycle 95000 ===
Register File:
x0 = 0x00000000
x1 = 0x00000002
x2 = 0x00000004
x3 = 0x00000008
x4 = 0x00000000
x5 = 0x00000000
x6 = 0x00000000
```

(a) jal x27, -4

```
=== Cycle 105000 ===
Register File:
x0 = 0x00000000
x1 = 0x00000002
x2 = 0x00000004
x3 = 0x00000008
x4 = 0x00000004
x5 = 0x00000000
x6 = 0x00000000
```

(b) jalr x28, -4(x1)

Figure 32: Jump Instruction Register Results

Part IV
Lab Part 4 – Hazard Handling

13 Introduction

Modern processor design employs pipelining to improve instruction throughput. Hazards are situations where the next instruction cannot execute in the following clock cycle. They are categorized into three types:

13.1 Data Hazards

Occur when an instruction depends on the result of a previous instruction still in the pipeline.

Example 1:

```
add  x1, x0, 2    // x1 = x0 + 2
add  x2, x1, x1   // x2 = x1 + x1
add  x3, x2, x2   // x3 = x2 + x2
sub  x4, x3, x2   // x4 = x3 - x2

0x00200093   // add x1, x0, 2
0x00108133   // add x2, x1, x1
0x002101b3   // add x3, x2, x2
0x40218233   // sub x4, x3, x2
```

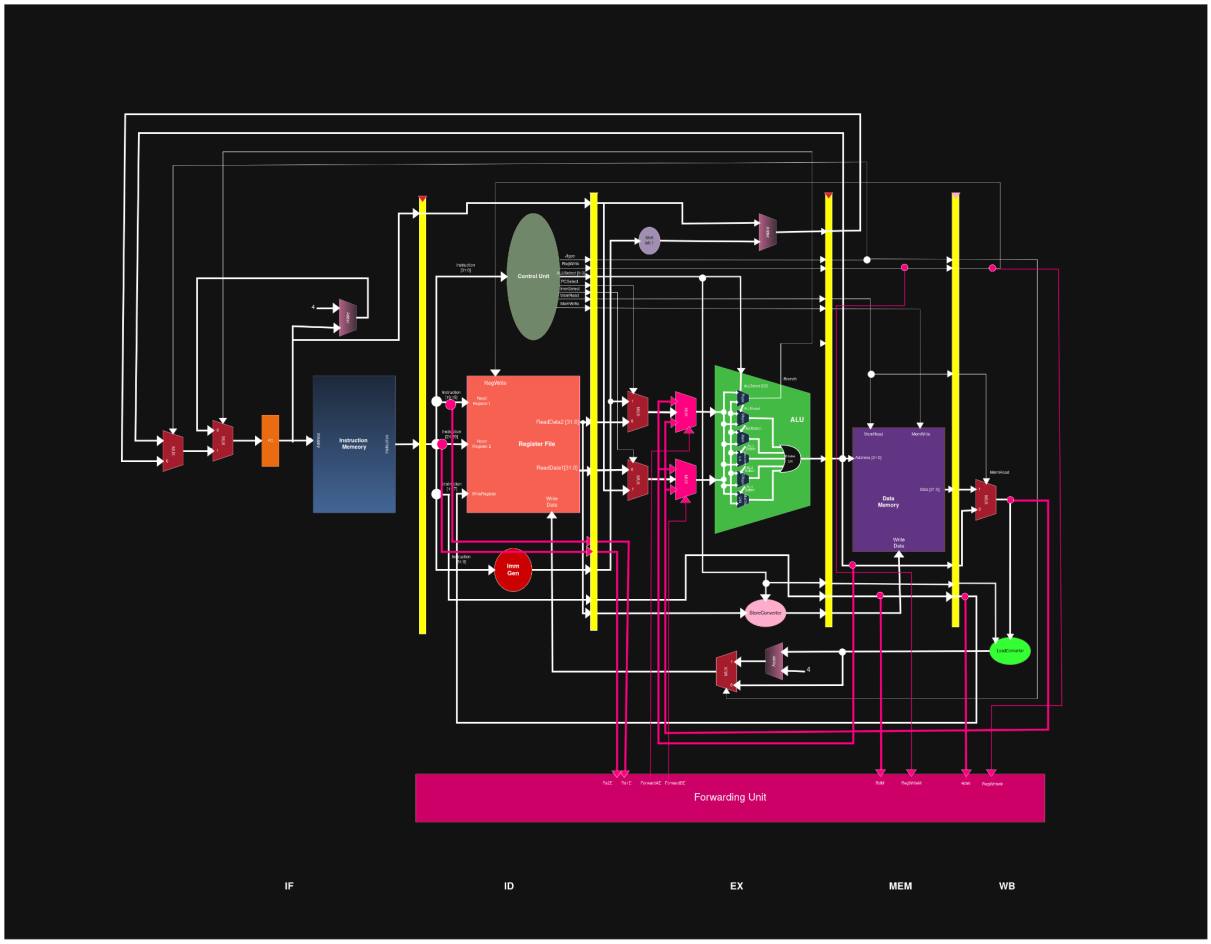


Figure 33: Data Hazard – Waveform Showing Incorrect Register Values Without Forwarding

x1 is updated to 2 but successor instructions do not update their registers correctly because the source registers have not been updated at the time those instructions fetch from the register file. This is a data hazard.

Example 2 – Load-Use Data Hazard:

```
0x00500093      // addi x1, x0, 5
0x00000000      // Nop
0x00000000      // Nop
0x00000000      // Nop
0x00000000      // Nop
0x0010a023      // sw x1, 0(x1)
0x0000a103      // lw x2, 0(x1)
0x00a10193      // addi x3, x2, 10
```

x3 is not updated properly (should be 15 = `0x0000F`, not 10 = `0x0000A`) because x2 had not been updated by the time the last instruction fetches data from the register file.

13.2 Control Hazards

Arise from branch and jump instructions whose target is not yet known.

Example 3:

```
00500093     // addi x1, x0, 5
00500113     // addi x2, x0, 5
00208663     // beq x1, x2, 12
00000193     // addi x3, x0, 0
0080006f     // jal x0, 8
00100193     // addi x3, x0, 1
00000013     // nop
```

After the 3rd instruction, it should jump to the 6th instruction. However, due to the control hazard, it proceeds to the 4th instruction because there is no mechanism to flush or stop wrongly fetched successor instructions.

13.3 Structural Hazards

Occur when hardware resources are insufficient to handle concurrent operations.

13.3.1 Causes

- Single memory used for both instructions and data.
- Limited number of ALUs, multipliers, or FPUs.
- Shared register file ports insufficient for simultaneous reads/writes.

13.3.2 Remedies and Solutions

1. **Hardware Duplication:** Use separate instruction and data memories (Harvard architecture). Add multiple ALUs, FPUs, or multipliers.
2. **Pipeline Scheduling:** Delay certain instructions intentionally to avoid resource conflicts.
3. **Stalling (Interlocking):** Insert NOPs or stalls until the resource becomes available.
4. **Resource Arbitration Logic:** Control logic to prioritize and manage access to shared resources dynamically.

To resolve these hazards, three specialized hardware modules were implemented:

1. **Forwarding Unit:** Mitigates data hazards by bypassing data from later stages directly to the execution stage.
2. **Hazard Detection Unit (HDU):** Detects load-use hazards and inserts pipeline stalls when necessary.
3. **Control Hazard Unit (CHU):** Detects control hazards due to branches and flushes incorrect instructions to redirect program flow.

14 Forwarding Unit

The Forwarding Unit detects data dependencies between instructions in different pipeline stages and forwards the required operand values from a later stage back to an earlier one before the write-back is complete.

14.1 How It Works

If the destination register of an instruction in the EX or MEM stage matches the source register (`rs1` or `rs2`) of the current instruction in the Decode or Execute stage, the unit enables forwarding paths to provide the correct operand directly from the pipeline register.

14.2 Result After Forwarding (Example 1)

```
0x00200093   // add x1, x0, 2
0x00108133   // add x2, x1, x1
0x002101b3   // add x3, x2, x2
0x40218233   // sub x4, x3, x2
```

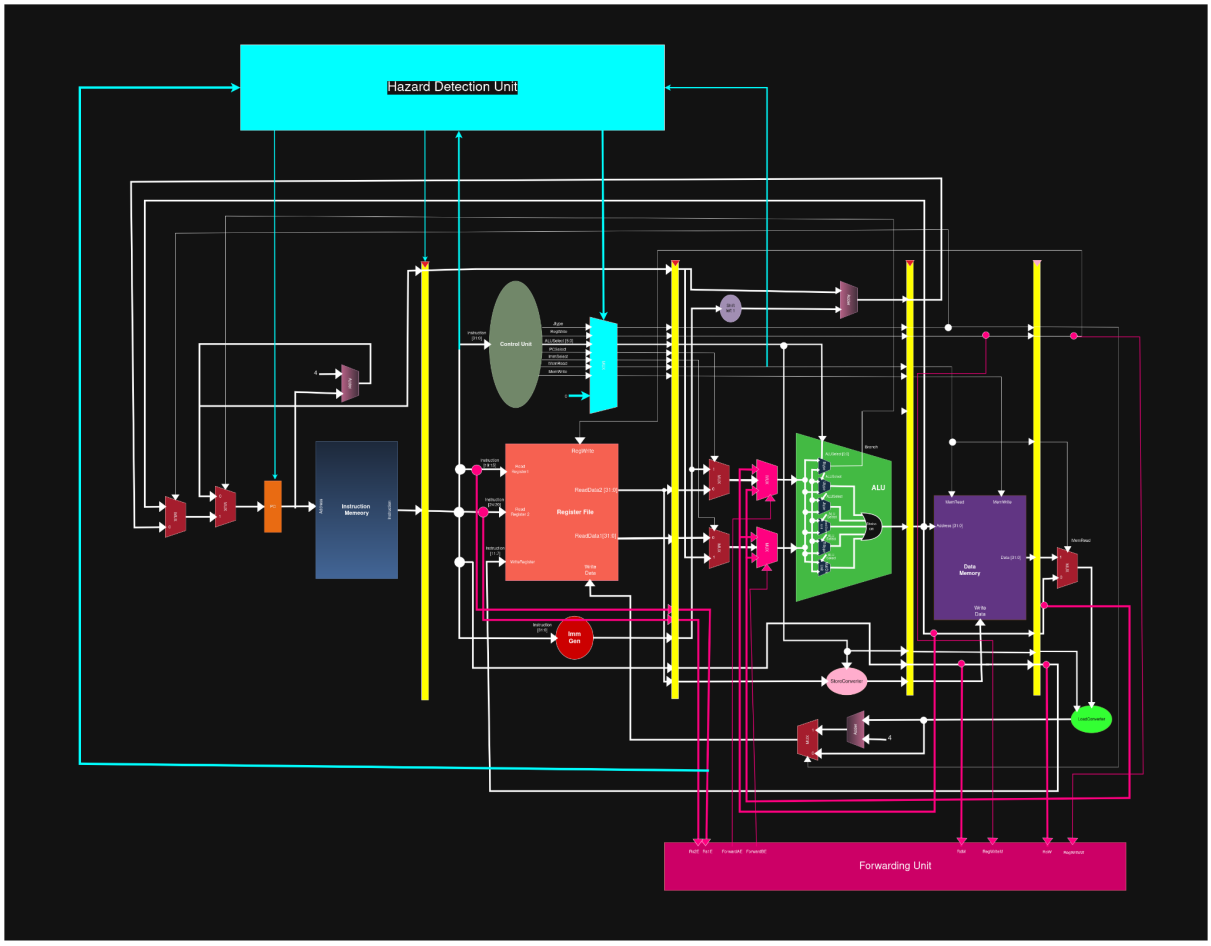


Figure 34: Data Hazard – Waveform After Forwarding Unit Applied (All Registers Correctly Updated)

Unlike in the example without forwarding, every instruction is now able to update the register values correctly.

15 Hazard Detection Unit

The Hazard Detection Unit (HDU) detects load-use hazards and inserts a stall (bubble) into the pipeline to ensure the required data is available.

15.1 How It Works

1. Checks if the instruction in the Execute stage is a load.
2. If the load destination (`rd`) matches the source register (`rs1` or `rs2`) of the instruction in the Decode stage.
3. Then: stall the pipeline, insert a NOP in the Execute stage, freeze the PC and IF/ID register.

15.2 Result

Prevents incorrect results, ensures data integrity, maintains correct program behavior.

15.3 Testing with Load-Use Hazard Example

```
0x00500093     // addi x1, x0, 5
0x00000000     // Nop
0x00000000     // Nop
0x00000000     // Nop
0x00000000     // Nop
0x0010a023     // sw x1, 0(x1)
0x0000a103     // lw x2, 0(x1)
0x00a10193     // addi x3, x2, 10
```

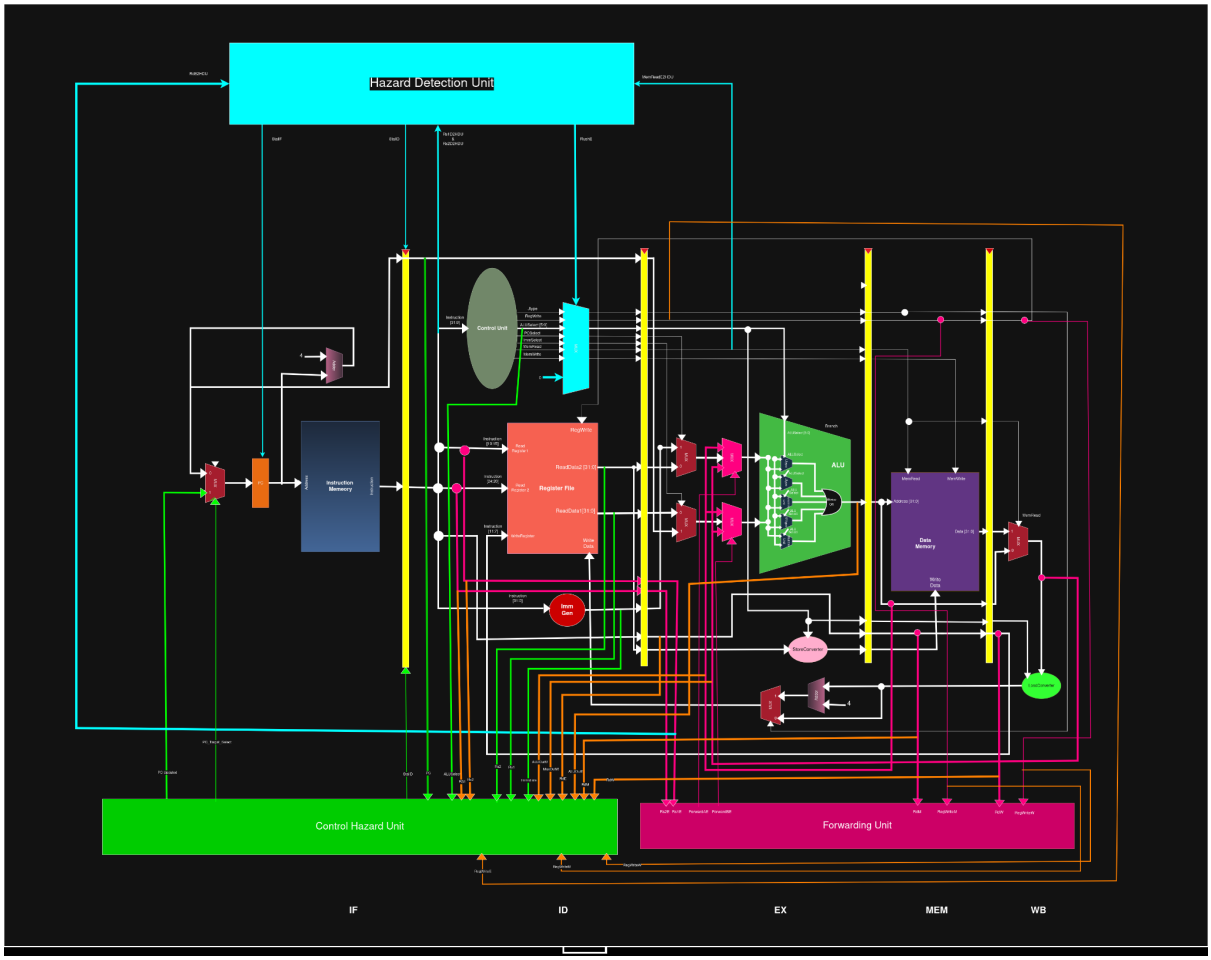


Figure 35: Load-Use Hazard – Waveform After HDU Applied (x3 = 15 = `0x0000000f`)

With the HDU applied, x3 is correctly updated by the end of 155 ns: x3 = x2 + 10 = 5 + 10 = 15 = `0x0000000f`.

16 Control Hazard Unit

A control hazard happens when the processor does not know the next instruction address because it is waiting to evaluate a branch or jump decision.

16.1 Purpose

The Control Hazard Unit (CHU) solves this by:

- Detecting when a branch or jump occurs.
- Deciding if the branch condition is true.
- Flushing the wrong instruction.
- Redirecting the PC to the target address if needed.

16.2 How It Works

In the Verilog module `Control_Hazard_Unit`:

1. Receives from the Decode stage: `aluSelect`, `rs1`, `rs2`, `imm`, `pc`.
2. Uses forwarding logic to get the most up-to-date values for `rs1` and `rs2`.
3. If the branch condition is true:
   - `FlushD = 1` → flushes the Decode stage.
   - `target_pc_valid = 1` → new branch address is ready.
   - `target_pc = pc + imm` (or `rs1 + imm` for JALR).

16.3 Outputs

- `FlushD` – Flushes the current instruction if the branch is taken.
- `target_pc_valid` – Indicates that a valid target PC is ready.
- `target_pc` – The address to jump to.

16.4 Testing with Control Hazard Example

```
00500093   // addi x1, x0, 5
00500113   // addi x2, x0, 5
00208663   // beq x1, x2, 12
00000193   // addi x3, x0, 0
0080006f   // jal x0, 8
00100193   // addi x3, x0, 1
00000013   // nop
```

Before the CHU, the 3rd instruction is succeeded by the 4th instruction without jumping to the 6th as required.

With the CHU applied, the branch and jump instructions correctly redirect the instruction flow: after implementing one instruction wrongly, the wrong instruction is flushed and the correct instruction is executed, as shown in the following waveform.

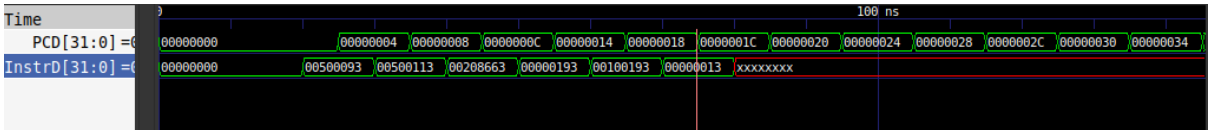


Figure 36: Control Hazard – Waveform After CHU Applied (Correct Branch/Jump Redirection)